학습 교안

파이썬 기초

처음 배우는 프로그래밍

변수·자료형부터 함수·클래스·파일·예외·모듈·제너레이터까지
12개 장 · 예제 140개 (실제 실행 결과 포함) · 확인 퀴즈 83문제

강사 김대철

학습 앱: https://sinaboro.github.io/python-learning-app/

이 교안의 사용법

이 교안은 파이썬을 처음 배우는 사람을 위해 만들었어요. 각 장은 **학습 목표 → 설명과 예제 → 총정리 퀴즈** 순서로 되어 있습니다. 예제는 모두 실제로 실행해서 얻은 결과를 실었어요. (random 처럼 실행할 때마다 달라지는 결과는 다를 수 있어요)

표시	뜻
파이썬 코드	직접 입력해 실행해 볼 코드 (파란 줄)
실행 결과	그 코드를 실제로 실행한 결과 (주황 줄). 입력이 필요한 예제는 사용한 입력값을 함께 적었어요
TIP	알아 두면 좋은 내용
주의	자주 하는 실수, 오류가 나는 경우
총정리 퀴즈	장 끝의 확인 문제. 정답과 해설은 맨 뒤에 있어요

학습 앱 교안과 똑같은 내용을 담은 학습 앱이 있어요. 스마트폰이나 PC 브라우저에서 https://sinaboro.github.io/python-learning-app/ 을 열면 예제 코드를 고쳐 가며 바로 실행하고, 퀴즈를 풀며 진도를 기록할 수 있어요. 카카오톡으로 링크를 받아 홈 화면에 설치하면 앱처럼 쓸 수 있습니다.

공부 순서 추천

① 설명을 읽고 → ② 예제를 **직접 타이핑**해 실행하고 → ③ 숫자나 글자를 바꿔 결과를 **예상한 뒤** 다시 실행하고 → ④ 장 끝 퀴즈로 확인하세요. 막히면 부록 A 의 방법으로 AI 에게 힌트를 요청해 보세요.

목차

이 교안의 사용법 . . . 2

01장 파이썬 시작하기 . . . 6

01.1 컴퓨터에게 일을 시키는 방법, 프로그래밍 . . . 6

01.2 왜 파이썬을 공부해야 할까? . . . 6

01.3 파이썬 실습을 위한 도구 준비하기 . . . 7

01.4 파이썬을 실행하는 다양한 방법 알아보기 . . . 7

01.5 파이썬 실행하기 전 알아둬야 할 3가지 . . . 8

02장 변수와 출력, 프로그래밍 기초 공부하기 . . . 10

02.1 변수 . . . 10

02.2 변수 활용하기 . . . 10

02.3 꼭 알아두면 좋은 프로그래밍 용어 12가지 . . . 11

02.4 변수·함수·클래스 이름 짓기 . . . 11

02.5 다양한 출력 연습하기 . . . 12

03장 기본 자료형 . . . 15

03.1 자료형과 데이터 . . . 15

03.2 숫자 . . . 15

03.3 문자열 . . . 16

03.4 문자열 응용과 자료형 변환 . . . 17

03.5 문자열 인덱싱과 슬라이싱 . . . 18

03.6 문자열 다양하게 다뤄보기 . . . 19

04장 컬렉션 자료형과 불리언 . . . 21

04.1 리스트 . . . 21

04.2 튜플 . . . 22

04.3 딕셔너리 . . . 22

04.4 집합 . . . 23

04.5 불리언 . . . 23

04.6 자료형 돌아보기 . . . 24

05장 흐름을 가르는 제어문, 조건문 . . . 26

05.1 if 문 . . . 26

05.2 비교 연산자, 논리 연산자 . . . 27

05.3 try~except 문 . . . 28

06장 반복을 만드는 제어문, 반복문 30

06.1 while 문과 반복문의 기초 30

06.2 for 문과 순회 31

06.3 리스트와 반복문 31

06.4 중첩 반복문 33

07장 코드를 묶어 재사용할 수 있는 함수 35

07.1 함수의 개념 알아보기 35

07.2 함수와 반환 알아보기 35

07.3 매개변수 좀 더 깊게 알아보기 36

07.4 함수를 한 줄로 정의할 수 있는 람다 37

07.5 함수와 스코프 37

07.6 재귀 함수 알아보기 38

07.7 알아두면 좋은 파이썬 내장 함수 6가지 39

08장 자료와 함수를 묶어 관리하는 클래스 41

08.1 클래스와 인스턴스 41

08.2 상속과 다형성 42

08.3 클래스 변수와 메서드 43

08.4 클래스 연습하기 44

09장 입력과 출력 47

09.1 출력이란 무엇일까? 47

09.2 파일 출력 이해하기 48

09.3 입력이란 무엇일까? 48

09.4 파일 입력 이해하기 49

09.5 파일을 한 줄씩 읽어 처리하기 50

09.6 메모장 프로그램 만들어보기 50

10장 오류 읽고 예외 처리하기 53

10.1 오류는 어떻게 읽고, 어떻게 처리해야 할까? 53

10.2 필요한 예외만 골라 처리하기 54

10.3 else 와 finally 로 실행 흐름 나누기 54

10.4 raise 로 직접 예외 발생시키기 55

10.5 오류에 대비한 메모장 프로그램 만들기 56

11장 코드를 나누고 가져오는 모듈과 라이브러리 59

11.1 모듈은 왜 필요할까? 59

11.2 표준 라이브러리 활용하기 . . . 59
11.3 모듈 만들고 사용해보기 . . . 60
11.4 패키지와 외부 라이브러리 알아보기 . . . 61
11.5 메모장 프로그램을 모듈로 나누기 . . . 61
11.6 모듈과 라이브러리 연습하기 . . . 62
12장 한 걸음 더 깊게 알아보는 파이썬 . . . 65
12.1 값을 복사한다는 것은 무엇일까? . . . 65
12.2 함수를 값처럼 다루기 . . . 66
12.3 반복의 정체, 이터러블과 제너레이터 . . . 67
부록 파이썬 실력을 완성하는 바로 쓰는 알찬 부록 . . . 70
부록 A. 인공지능과 함께 파이썬 공부하기 . . . 70
부록 B. 자주 쓰는 내장 함수 25가지 . . . 70
부록 C. 자주 쓰는 표준 라이브러리 12가지 . . . 71
부록 D. 종합 실력 점검 20문제 . . . 72
정답과 해설 . . . 76

CHAPTER 01

파이썬 시작하기

학습 목표

- 프로그래밍이 무엇인지 말로 설명할 수 있다.
- 파이썬을 배우면 좋은 이유를 안다.
- 파이썬과 VS Code를 설치하고 첫 프로그램을 실행할 수 있다.
- 대화형 셸, 파일 실행, 노트북 세 가지 실행 방법을 구분한다.

01.1 컴퓨터에게 일을 시키는 방법, 프로그래밍

컴퓨터는 아주 빠르고 정확하지만, **스스로 생각하지는 못합니다.** 무엇을, 어떤 순서로 할지 하나하나 알려 줘야 해요. 이렇게 컴퓨터에게 내리는 명령의 목록을 **프로그램**이라 하고, 프로그램을 만드는 일을 **프로그래밍(코딩)**이라고 합니다.

라면 끓이기를 예로 들어 볼게요. 사람에게는 "라면 끓여 줘" 한마디면 되지만, 컴퓨터에게는 순서를 빠짐없이 알려 줘야 합니다.

1. 냄비에 물 550mL를 붓는다.
2. 물이 끓을 때까지 불을 켠다.
3. 면과 스프를 넣는다.
4. 4분 30초 기다린 뒤 불을 끈다.

이렇게 일을 처리하는 **순서와 방법**을 **알고리즘**이라고 해요. 프로그래밍은 알고리즘을 컴퓨터가 알아듣는 말, 즉 **프로그래밍 언어**로 옮겨 적는 일입니다.

파이썬 코드

```
# 컴퓨터에게 내리는 첫 번째 명령: 화면에 글자를 보여 줘!
print("안녕하세요, 파이썬!")
```

실행 결과

```
안녕하세요, 파이썬!
```

TIP `print()` 는 괄호 안의 내용을 화면에 보여 주는 명령이에요. 앞으로 가장 많이 쓰게 될 친구입니다.

01.2 왜 파이썬을 공부해야 할까?

프로그래밍 언어는 C, Java, JavaScript 등 아주 많아요. 그중 파이썬(Python)은 처음 배우기에 특히 좋습니다.

- **문법이 쉽고 짧아요.** 영어 문장처럼 읽혀서 처음 보는 사람도 뜻을 짐작할 수 있어요.
- **쓰이는 곳이 넓어요.** 인공지능·데이터 분석, 웹 서비스, 업무 자동화, 게임까지 만들 수 있어요.
- **도구가 풍부해요.** 다른 사람이 만들어 둔 라이브러리를 가져다 쓰면 어려운 일도 몇 줄로 끝나요.
- **함께 배우는 사람이 많아요.** 모르는 것을 검색하면 답을 쉽게 찾을 수 있어요.

같은 일을 하는 코드를 다른 언어와 비교해 보면 파이썬이 얼마나 간단한지 알 수 있어요. 아래는 1부터 5까지 더하는 파이썬 코드입니다.

파이썬 코드

```
total = 0
for n in range(1, 6):   # 1, 2, 3, 4, 5 를 차례로 꺼내서
    total = total + n   # 하나씩 더한다
print("1부터 5까지의 합:", total)
```

실행 결과

```
1부터 5까지의 합: 15
```

TIP 지금은 코드를 다 이해하지 못해도 괜찮아요. "이렇게 짧게 쓸 수 있구나" 정도만 느끼면 충분합니다.

01.3 파이썬 실습을 위한 도구 준비하기

파이썬 코드를 실행하려면 두 가지가 필요해요.

도구	하는 일	비유
파이썬 (인터프리터)	코드를 읽고 실행한다	요리사
코드 편집기 (VS Code)	코드를 쓰고 저장한다	요리책을 쓰는 노트

파이썬 설치 (윈도우)

1. python.org 에 접속해 **Downloads** 에서 최신 버전(3.12 이상)을 내려받습니다.
2. 설치 첫 화면 아래의 **Add python.exe to PATH** 를 꼭 체크합니다.
3. **Install Now** 를 누릅니다.
4. 명령 프롬프트(cmd)를 열고 `python --version` 을 입력해 버전이 나오면 성공입니다.

화면

```
C:\> python --version
Python 3.12.0
```

주의 **Add python.exe to PATH** 를 체크하지 않으면 명령 프롬프트에서 `python` 명령을 찾지 못해요. 놓쳤다면 설치 파일을 다시 실행해 **Modify** 로 고칠 수 있어요.

VS Code 설치

1. code.visualstudio.com 에서 내려받아 설치합니다.
2. 왼쪽 **확장(Extensions)** 메뉴에서 **Python** 을 검색해 설치합니다. (Microsoft 제작)
3. 한국어 화면이 좋다면 **Korean Language Pack** 도 설치하세요.

TIP 설치가 어렵다면 이 교안과 함께 제공되는 **학습 앱**에서 바로 코드를 실행해 볼 수 있어요. 처음 한 번만 인터넷이 필요합니다.

01.4 파이썬을 실행하는 다양한 방법 알아보기

파이썬 코드를 실행하는 방법은 크게 세 가지예요.

① 대화형 셸 (한 줄씩 바로 실행)

명령 프롬프트에서 `python` 을 입력하면 `>>>` 표시가 나와요. 여기에 코드를 한 줄 쓰고 Enter 를 누르면 바로 결과가 나옵니다. 계산기처럼 간단히 확인할 때 좋아요. 끝낼 때는 `exit()` 를 입력합니다.

```
화면
>>> 3 + 4
7
>>> print("반가워요")
반가워요
>>> exit()
```

② 파일로 저장해서 실행

VS Code 에서 `hello.py` 처럼 이름 끝이 **.py** 인 파일을 만들고 코드를 저장한 뒤, 오른쪽 위 ▷**(실행)** 버튼을 누르거나 터미널에서 `python hello.py` 를 입력합니다. 여러 줄짜리 프로그램은 이렇게 만들어요.

```
파이썬 코드
# hello.py – 파일로 저장해서 실행하는 첫 프로그램
print("첫 번째 줄")
print("두 번째 줄")
print("파일은 위에서 아래로 차례대로 실행돼요")
```

```
실행 결과
첫 번째 줄
두 번째 줄
파일은 위에서 아래로 차례대로 실행돼요
```

③ 노트북 (주피터·코랩)

코드를 **칸(셀)** 단위로 나눠 실행하고, 결과를 바로 아래에서 보는 방식이에요. 구글 코랩(colab.research.google.com)은 설치 없이 웹 브라우저에서 쓸 수 있어 데이터 분석·인공지능 공부에 많이 쓰입니다.

방법	장점	쓰기 좋은 때
대화형 셸	바로 결과 확인	짧은 계산, 기능 시험
파일 실행	저장·재사용	프로그램 만들기
노트북	칸마다 결과 확인	데이터 분석, 수업 실습

01.5 파이썬 실행하기 전 알아둬야 할 3가지

① 위에서 아래로, 한 줄씩

파이썬은 코드를 **맨 위부터 한 줄씩** 순서대로 실행해요. 순서를 바꾸면 결과도 바뀝니다.

```
파이썬 코드
print("1. 일어난다")
print("2. 세수한다")
print("3. 학교에 간다")
```

```
실행 결과
1. 일어난다
2. 세수한다
3. 학교에 간다
```

② 대소문자와 기호를 정확히

`print` 를 `Print` 로 쓰면 다른 이름이 되어 오류가 나요. 따옴표와 괄호도 짝을 맞춰야 합니다. 기호는 반드시 **영어 자판(반각)** 으로 입력하세요.

```
파이썬 코드
Print("안녕")    # P 를 대문자로 쓰면?
```

실행 결과

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    Print("안녕")    # P 를 대문자로 쓰면?
NameError: name 'Print' is not defined
```

TIP 오류 메시지의 마지막 줄을 읽어 보세요. `NameError: name 'Print' is not defined` 는 "Print 라는 이름을 모른다"는 뜻이에요. 오류는 실패가 아니라 **컴퓨터가 주는 힌트**입니다.

③ # 뒤는 주석 (사람을 위한 메모)

`#` 뒤에 쓴 글은 실행되지 않아요. 코드가 하는 일을 설명할 때 씁니다. 나중의 나와 동료를 위해 주석을 다는 습관을 들이세요.

파이썬 코드

```
# 이 줄은 실행되지 않아요
print("주석 다음 줄은 실행돼요")  # 줄 끝에 쓴 주석도 무시돼요
```

실행 결과

```
주석 다음 줄은 실행돼요
```

총정리 퀴즈

Q1. 컴퓨터에게 일을 처리하는 순서와 방법을 정리한 것을 무엇이라 하나요?

① 컴파일러
② 알고리즘
③ 인터프리터
④ 라이브러리

Q2. 화면에 글자를 보여 주는 파이썬 명령은?

① show()
② echo()
③ print()
④ write()

Q3. 윈도우에 파이썬을 설치할 때 꼭 체크해야 하는 항목은?

① Add python.exe to PATH
② Install for all users
③ Disable path length limit
④ Install launcher

Q4. 파이썬 파일의 확장자(이름 끝)는?

① .txt
② .pyt
③ .py
④ .exe

Q5. 다음 중 실행되지 않는 줄은?

① print("A")
② # print("B")
③ print("#C")
④ print('D')

CHAPTER 02

변수와 출력, 프로그래밍 기초 공부하기

학습 목표

- 변수를 만들고 값을 저장·변경할 수 있다.
- 변수로 계산하고 값을 서로 바꿀 수 있다.
- 프로그래밍 기본 용어 12가지를 설명할 수 있다.
- 좋은 이름 짓기 규칙을 지킬 수 있다.
- print() 의 여러 출력 방법(sep, end, f-문자열)을 쓸 수 있다.

02.1 변수

변수는 값을 담아 두는 **이름표 붙은 상자**예요. = 기호로 상자에 값을 넣습니다. 수학의 "같다"가 아니라 **"오른쪽 값을 왼쪽 이름에 넣어라"**는 뜻이에요.

파이썬 코드

```
age = 20              # age 라는 상자에 20 을 넣는다
name = "김파이"        # 글자(문자열)도 담을 수 있다
print(name, age)
```

실행 결과

```
김파이 20
```

변수에는 언제든 새 값을 넣을 수 있어요. 그러면 예전 값은 사라지고 새 값으로 바뀝니다.

파이썬 코드

```
score = 70
print("처음 점수:", score)
score = 95           # 새 값으로 바꾸기
print("바뀐 점수:", score)
```

실행 결과

```
처음 점수: 70
바뀐 점수: 95
```

TIP 값을 넣지 않은 변수를 쓰면 `NameError` 가 나요. 변수는 **먼저 만들고 나중에 사용**해야 합니다.

02.2 변수 활용하기

변수끼리 계산하고, 계산 결과를 다시 변수에 담을 수 있어요.

파이썬 코드

```
price = 4500         # 커피 한 잔 가격
count = 3            # 몇 잔
total = price * count
print("총 금액:", total, "원")
```

실행 결과

```
총 금액: 13500 원
```

자기 자신을 이용해 값 바꾸기

`x = x + 1` 은 "x 에 1 을 더한 값을 다시 x 에 넣어라"예요. 줄여서 `x += 1` 로 쓸 수 있어요.

파이썬 코드

```
step = 0
step = step + 1      # 1
step += 1            # 2 (위와 같은 뜻을 짧게)
step *= 10           # 20 (step = step * 10)
print(step)
```

실행 결과

```
20
```

여러 변수에 한 번에 넣기, 값 바꾸기

파이썬 코드

```
a, b = 1, 2          # a 에 1, b 에 2
print("바꾸기 전:", a, b)
a, b = b, a          # 파이썬은 한 줄로 두 값을 바꿀 수 있다
print("바꾼 뒤:", a, b)
```

실행 결과

```
바꾸기 전: 1 2
바꾼 뒤: 2 1
```

02.3 꼭 알아두면 좋은 프로그래밍 용어 12가지

앞으로 계속 나올 말들이에요. 지금은 "이런 말이 있구나" 정도로 훑어보고, 모르면 다시 돌아와 찾아보세요.

용어	뜻	예
코드	프로그래밍 언어로 쓴 글	print("hi")
실행	코드를 컴퓨터가 처리하는 것	▷ 버튼
변수	값을 담는 이름표 붙은 상자	age = 20
값	변수에 담기는 데이터	20, "hi"
자료형	값의 종류	정수, 문자열
연산자	계산·비교에 쓰는 기호	+ - * / ==
식(표현식)	계산되어 값이 되는 코드	3 + 4
문(문장)	실행되는 명령 한 줄	x = 3
함수	이름으로 불러 쓰는 기능 묶음	print(), len()
인수	함수에 넘겨주는 값	print("hi") 의 "hi"
반환값	함수가 돌려주는 결과	len("abc") → 3
오류(에러)	실행이 멈추는 문제	NameError

파이썬 코드

```
# 용어를 코드에서 찾아보기
word = "python"          # 문: word 라는 변수에 값 "python" 을 넣는다
length = len(word)       # 함수 len 에 인수 word 를 넘기고, 반환값 6 을 받는다
print(length * 2)        # 식 length * 2 가 계산되어 12 가 된다
```

실행 결과

```
12
```

02.4 변수·함수·클래스 이름 짓기

꼭 지켜야 하는 규칙 (안 지키면 오류)

- 영문자, 숫자, 밑줄(_)만 쓸 수 있어요. (한글도 되지만 권장하지 않아요)
- **숫자로 시작할 수 없어요.** `1st` (X), `first` (O)
- 띄어쓰기를 할 수 없어요. `my name` (X), `my_name` (O)
- `if`, `for`, `class` 같은 **예약어**는 이름으로 쓸 수 없어요.

파이썬 코드 (실행하지 않는 예시)
```
1st_place = "금메달"   # 숫자로 시작 → SyntaxError
for = 3              # 예약어 → SyntaxError
```

지키면 좋은 약속 (파이썬 스타일, PEP 8)

대상	방식	예
변수·함수	소문자 + 밑줄 (snake_case)	user_name, get_total
클래스	단어 첫 글자 대문자 (PascalCase)	BankAccount
상수	모두 대문자	MAX_SIZE

파이썬 코드
```
MAX_SCORE = 100                # 바뀌지 않을 값은 대문자로
student_name = "이코딩"          # 무엇이 담겼는지 알 수 있는 이름
math_score = 88
print(student_name, "의 수학 점수는", math_score, "/", MAX_SCORE)
```

실행 결과
```
이코딩 의 수학 점수는 88 / 100
```

TIP `a`, `x2`, `temp` 보다 `price`, `user_count` 처럼 **뜻이 드러나는 이름**이 좋아요. 이름만 봐도 무엇이 담겼는지 알 수 있어야 합니다.

02.5 다양한 출력 연습하기

쉼표로 여러 값 출력하기

`print()` 에 쉼표로 여러 값을 넣으면 **띄어쓰기 한 칸**으로 이어서 출력해요.

파이썬 코드
```
print("사과", 3, "개")
```

실행 결과
```
사과 3 개
```

sep: 사이에 넣을 글자, end: 끝에 붙일 글자

파이썬 코드
```
print("2026", "11", "04", sep="-")    # 사이를 - 로
print("한 줄에", end=" ")              # 줄을 바꾸지 않고 띄어쓰기만
print("이어서 출력")
```

실행 결과
```
2026-11-04
한 줄에 이어서 출력
```

f-문자열: 글자 안에 변수 넣기

따옴표 앞에 `f` 를 붙이고 `{변수}` 를 쓰면 그 자리에 값이 들어가요. 가장 많이 쓰는 방법입니다.

파이썬 코드
```
name = "김파이"
age = 20
print(f"제 이름은 {name}이고, 나이는 {age}살입니다.")
print(f"내년에는 {age + 1}살이 돼요.")        # 중괄호 안에서 계산도 된다
print(f"원주율은 약 {3.14159:.2f}입니다.")    # :.2f → 소수점 둘째 자리까지
```

실행 결과
```
제 이름은 김파이이고, 나이는 20살입니다.
내년에는 21살이 돼요.
원주율은 약 3.14입니다.
```

특수 문자: ₩n 줄바꿈, ₩t 탭

파이썬 코드
```
print("첫째 줄\n둘째 줄")
print("이름\t점수")
print("김파이\t95")
```

실행 결과
```
첫째 줄
둘째 줄
이름    점수
김파이    95
```

총정리 퀴즈

Q1. age = 20 에서 = 의 뜻은?

① age 와 20 이 같은지 비교한다
② 오른쪽 값 20 을 왼쪽 변수 age 에 넣는다
③ age 를 지운다
④ 20 을 출력한다

Q2. x = 5 다음에 x += 3 을 실행하면 x 는?

① 3
② 5
③ 8
④ 15

Q3. 변수 이름으로 쓸 수 없는 것은?

① user_name
② _count
③ 2nd_place
④ totalScore

Q4. print("A", "B", sep="/") 의 결과는?

① A B
② A/B
③ AB
④ A/B/

Q5. name = "파이" 일 때 print(f"안녕 {name}") 의 결과는?

① 안녕 {name}
② 안녕 파이
③ f안녕 파이
④ 오류

CHAPTER 03

기본 자료형

학습 목표

- 자료형이 무엇인지, type() 으로 확인하는 법을 안다.
- 정수·실수 계산과 연산자 우선순위를 이해한다.
- 문자열을 만들고 더하기·곱하기·길이 구하기를 할 수 있다.
- 자료형을 서로 바꿀 수 있다 (int, float, str).
- 인덱싱과 슬라이싱으로 문자열 일부를 꺼낼 수 있다.
- 자주 쓰는 문자열 메서드를 쓸 수 있다.

03.1 자료형과 데이터

프로그램이 다루는 값(데이터)에는 **종류**가 있어요. 이것을 **자료형(type)**이라고 합니다. 숫자는 더할 수 있고, 글자는 붙일 수 있듯이 자료형에 따라 할 수 있는 일이 달라요. `type()` 으로 값의 자료형을 확인할 수 있습니다.

자료형	이름	예
int	정수	10, -3, 0
float	실수(소수)	3.14, -0.5
str	문자열(글자)	"hello", '파이썬'
bool	불리언(참·거짓)	True, False

파이썬 코드

```
print(type(10))
print(type(3.14))
print(type("안녕"))
print(type(True))
```

실행 결과

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

03.2 숫자

사칙연산과 여러 연산자

연산자	뜻	예	결과
+	더하기	7 + 2	9
-	빼기	7 - 2	5
*	곱하기	7 * 2	14
/	나누기 (항상 실수)	7 / 2	3.5
//	몫	7 // 2	3
%	나머지	7 % 2	1
**	거듭제곱	7 ** 2	49

파이썬 코드

```
print(7 / 2, 7 // 2, 7 % 2)
print(2 ** 10)          # 2 를 10번 곱하기
print(10 / 5)           # 나누어떨어져도 / 의 결과는 실수
```

실행 결과

```
3.5 3 1
1024
2.0
```

TIP `%` (나머지)는 짝수·홀수 판별에 자주 써요. `n % 2 == 0` 이면 짝수입니다.

연산 순서

수학처럼 `**` → `* / // %` → `+ -` 순서로 계산해요. 헷갈리면 **괄호**를 쓰세요.

파이썬 코드

```
print(2 + 3 * 4)        # 곱하기 먼저 → 14
print((2 + 3) * 4)      # 괄호 먼저 → 20
```

실행 결과

```
14
20
```

실수 계산의 작은 오차

컴퓨터는 실수를 2진수로 저장해서 아주 작은 오차가 생길 수 있어요. 출력할 때 `round()` 로 반올림하면 됩니다.

파이썬 코드

```
print(0.1 + 0.2)
print(round(0.1 + 0.2, 2))   # 소수점 둘째 자리까지 반올림
```

실행 결과

```
0.30000000000000004
0.3
```

03.3 문자열

글자들의 줄을 **문자열**이라고 해요. 큰따옴표(" ")나 작은따옴표(' ')로 감쌉니다. 여러 줄은 따옴표 세 개(""" """)로 감싸요.

파이썬 코드

```
a = "큰따옴표 문자열"
b = '작은따옴표 문자열'
c = """여러 줄을
그대로 쓸 수 있어요"""
print(a); print(b); print(c)
```

실행 결과

```
큰따옴표 문자열
작은따옴표 문자열
여러 줄을
그대로 쓸 수 있어요
```

글자 안에 따옴표를 넣고 싶으면 다른 종류의 따옴표로 감싸거나, 앞에 `\` 를 붙여요.

파이썬 코드
```
print("He said 'Hi'")
print('It\'s OK')
```

실행 결과
```
He said 'Hi'
It's OK
```

문자열 더하기·곱하기·길이

파이썬 코드
```
first = "파이"
last = "썬"
print(first + last)        # 이어 붙이기
print("=" * 20)            # 20번 반복
print(len("Hello"))        # 글자 수
```

실행 결과
```
파이썬
====================
5
```

03.4 문자열 응용과 자료형 변환

숫자와 문자열은 바로 더할 수 없어요

파이썬 코드
```
age = 20
print("나이: " + age)      # 문자열 + 정수 → 오류
```

실행 결과
```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    print("나이: " + age)      # 문자열 + 정수 → 오류
TypeError: can only concatenate str (not "int") to str
```

`TypeError` 는 "자료형이 맞지 않는다"는 뜻이에요. **자료형 변환** 함수로 맞춰 줍니다.

함수	바꾸는 자료형	예	결과
int()	정수	int("25"), int(3.9)	25, 3 (버림)
float()	실수	float("2.5")	2.5
str()	문자열	str(100)	"100"

파이썬 코드
```
age = 20
print("나이: " + str(age))   # 숫자를 문자열로
year = "2026"
print(int(year) + 1)         # 문자열을 숫자로
print(int(3.9))              # 실수 → 정수는 소수점 아래를 버린다
```

실행 결과
```
나이: 20
2027
3
```

TIP `input()` 으로 받은 값은 **항상 문자열**이에요. 숫자로 계산하려면 `int(input())` 처럼 바꿔야 합니다. (9장에서 자세히 배워요)

파이썬 코드
```
n = input("숫자를 입력하세요: ")
print("두 배:", int(n) * 2)
```

실행 결과 · 입력값: 17
```
숫자를 입력하세요: 17
두 배: 34
```

03.5 문자열 인덱싱과 슬라이싱

문자열의 글자에는 **번호(인덱스)**가 있어요. 맨 앞이 **0번**이고, 뒤에서부터는 **-1, -2 …** 로 셀 수 있습니다.

글자	P	y	t	h	o	n
앞에서	0	1	2	3	4	5
뒤에서	-6	-5	-4	-3	-2	-1

파이썬 코드
```
s = "Python"
print(s[0])      # 첫 글자
print(s[-1])     # 마지막 글자
print(s[2])
```

실행 결과
```
P
n
t
```

슬라이싱: [시작:끝] — 끝 번호는 포함하지 않아요

파이썬 코드
```
s = "Python"
print(s[0:2])    # 0, 1 번 → Py
print(s[2:])     # 2번부터 끝까지
print(s[:4])     # 처음부터 3번까지
print(s[::-1])   # 거꾸로 (간격 -1)
```

실행 결과
```
Py
thon
Pyth
nohtyP
```

파이썬 코드
```
date = "20261104"
print(date[:4] + "년", date[4:6] + "월", date[6:] + "일")
```

실행 결과
```
2026년 11월 04일
```

주의 문자열은 **바꿀 수 없는(immutable)** 자료형이에요. `s[0] = "J"` 처럼 글자 하나를 바꾸면 오류가 나요. 새 문자열을 만들어야 합니다.

03.6 문자열 다양하게 다뤄보기

문자열 뒤에 . 을 찍고 쓰는 기능을 **메서드**라고 해요. 원래 문자열은 그대로 두고 **새 문자열을 돌려줍니다.**

메서드	하는 일	예	결과
upper() / lower()	대문자 / 소문자로	"Hi".upper()	"HI"
strip()	양쪽 공백 지우기	" hi ".strip()	"hi"
replace(a, b)	a 를 b 로 바꾸기	"a-b".replace("-", "+")	"a+b"
split(구분자)	나눠서 리스트로	"a,b".split(",")	['a', 'b']
join(리스트)	이어 붙이기	"-".join(['a','b'])	"a-b"
find(글자)	위치 찾기 (없으면 -1)	"hello".find("l")	2
count(글자)	개수 세기	"banana".count("a")	3
startswith()	~로 시작하나	"file.py".startswith("file")	True

파이썬 코드

```
text = "  Hello Python  "
print(text.strip())
print(text.strip().upper())
print("사과,배,포도".split(","))
print(" / ".join(["월", "화", "수"]))
print("banana".count("a"), "banana".find("n"))
print("I like Java".replace("Java", "Python"))
```

실행 결과

```
Hello Python
HELLO PYTHON
['사과', '배', '포도']
월 / 화 / 수
3 2
I like Python
```

in: 들어 있는지 확인

파이썬 코드

```
email = "user@example.com"
print("@" in email)        # True
print(email.endswith(".com"))
```

실행 결과

```
True
True
```

총정리 퀴즈

Q1. type(3.0) 의 결과는?

① int
② float
③ str
④ bool

Q2. 7 // 2 와 7 % 2 의 결과는?

① 3.5, 1
② 3, 1
③ 3, 0
④ 4, 1

Q3. "ab" * 3 의 결과는?

① ab3
② abababab
③ 오류
④ a b a b a b

Q4. s = "Python" 일 때 s[1:4] 는?

① Pyt
② yth
③ ytho
④ Pyth

Q5. "나이: " + 20 을 실행하면?

① 나이: 20
② TypeError (문자열과 정수는 더할 수 없음)
③ 나이: + 20
④ 20

Q6. "a,b,c".split(",") 의 결과는?

① "abc"
② ['a', 'b', 'c']
③ ('a', 'b', 'c')
④ a b c

CHAPTER 04

컬렉션 자료형과 불리언

학습 목표

- 리스트를 만들고 추가·삭제·정렬할 수 있다.
- 튜플이 리스트와 어떻게 다른지 안다.
- 딕셔너리로 키와 값을 저장하고 꺼낼 수 있다.
- 집합으로 중복을 없애고 교집합·합집합을 구할 수 있다.
- 불리언(True/False)과 참·거짓으로 취급되는 값을 안다.

04.1 리스트

여러 값을 **순서대로** 한곳에 담는 자료형이에요. 대괄호 [ ] 안에 쉼표로 구분해 씁니다. 문자열처럼 인덱싱·슬라이싱이 되고, **값을 바꿀 수 있어요.**

파이썬 코드

```
fruits = ["사과", "바나나", "포도"]
print(fruits[0], fruits[-1])
print(fruits[1:])
fruits[1] = "딸기"          # 값 바꾸기
print(fruits, len(fruits))
```

실행 결과

```
사과 포도
['바나나', '포도']
['사과', '딸기', '포도'] 3
```

추가·삭제

메서드	하는 일
append(x)	맨 뒤에 x 추가
insert(i, x)	i 번 자리에 x 넣기
remove(x)	값 x 를 찾아 지우기 (첫 번째 하나)
pop() / pop(i)	맨 뒤(또는 i 번) 값을 꺼내며 지우기
del 리스트[i]	i 번 지우기

파이썬 코드

```
nums = [10, 20, 30]
nums.append(40)
nums.insert(0, 5)
print(nums)
nums.remove(20)
last = nums.pop()
print(nums, "꺼낸 값:", last)
```

실행 결과

```
[5, 10, 20, 30, 40]
[5, 10, 30] 꺼낸 값: 40
```

정렬과 여러 기능

파이썬 코드

```
scores = [70, 95, 82, 60]
scores.sort()                    # 오름차순으로 제자리 정렬
print(scores)
scores.sort(reverse=True)        # 내림차순
print(scores)
print(max(scores), min(scores), sum(scores))
print(sorted([3, 1, 2]))         # 원본은 두고 정렬된 새 리스트
print(82 in scores, scores.index(82))
```

실행 결과

```
[60, 70, 82, 95]
[95, 82, 70, 60]
95 60 307
[1, 2, 3]
True 1
```

TIP 리스트 안에 리스트를 넣을 수도 있어요. `[[1, 2], [3, 4]]` 에서 `[1][0]` 은 3 이에요. 표 같은 데이터를 담을 때 씁니다.

04.2 튜플

튜플은 소괄호 `( )` 로 만드는 **바꿀 수 없는 리스트**예요. 한 번 정하면 바뀌면 안 되는 값(좌표, 생년월일 등)을 담을 때 써요.

파이썬 코드

```
point = (3, 5)
print(point[0], point[1])
x, y = point                # 튜플 풀기(언패킹)
print(f"x={x}, y={y}")
```

실행 결과

```
3 5
x=3, y=5
```

파이썬 코드

```
point = (3, 5)
point[0] = 10               # 튜플은 바꿀 수 없다 → 오류
```

실행 결과

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    point[0] = 10               # 튜플은 바꿀 수 없다 → 오류
TypeError: 'tuple' object does not support item assignment
```

TIP 값이 하나인 튜플은 `(5,)` 처럼 쉼표를 붙여야 해요. `(5)` 는 그냥 숫자 5 입니다.

04.3 딕셔너리

키(key)와 값(value)을 짝지어 저장하는 자료형이에요. 중괄호 `{ }` 안에 `키: 값` 을 씁니다. 사전에서 단어(키)로 뜻(값)을 찾는 것과 같아요.

파이썬 코드

```
student = {"name": "김파이", "age": 20, "major": "컴퓨터"}
print(student["name"])
student["age"] = 21              # 값 바꾸기
student["email"] = "pi@ex.com"   # 새 키 추가
del student["major"]             # 삭제
print(student)
```

실행 결과

```
김파이
{'name': '김파이', 'age': 21, 'email': 'pi@ex.com'}
```

안전하게 꺼내기, 모두 돌아보기

파이썬 코드

```
student = {"name": "김파이", "age": 20}
print(student.get("phone"))            # 없는 키 → None (오류 없음)
print(student.get("phone", "없음"))     # 없을 때 대신 쓸 값
print(student.keys())
print(student.values())
print(student.items())
print("name" in student)
```

실행 결과

```
None
없음
dict_keys(['name', 'age'])
dict_values(['김파이', 20])
dict_items([('name', '김파이'), ('age', 20)])
True
```

주의 `student["phone"]` 처럼 없는 키를 대괄호로 꺼내면 `KeyError` 가 나요. 있는지 모를 때는 `get()` 을 쓰세요.

04.4 집합

집합(set)은 **중복이 없고 순서도 없는** 자료형이에요. 중복 제거와 교집합·합집합 계산에 편리합니다.

파이썬 코드

```
nums = [1, 2, 2, 3, 3, 3]
print(set(nums))                 # 중복이 사라진다
a = {"파이썬", "자바", "C"}
b = {"파이썬", "자바스크립트"}
print(a & b)                     # 교집합
print(len(a | b))                # 합집합의 크기
print(a - b)                     # 차집합
```

실행 결과

```
{1, 2, 3}
{'파이썬'}
4
{'C', '자바'}
```

TIP 집합은 순서가 없어서 출력 순서가 실행할 때마다 다를 수 있고, 인덱스(`[0]`)로 꺼낼 수 없어요.

04.5 불리언

참(True)과 거짓(False) 두 값만 있는 자료형이에요. 비교 결과가 불리언으로 나옵니다. (첫 글자는 대문자!)

파이썬 코드
```
print(10 > 3)
print(10 == 3)
print(type(True))
```

실행 결과
```
True
False
<class 'bool'>
```

참·거짓으로 취급되는 값

`bool()` 로 확인해 보면, **비어 있거나 0인 값은 거짓**, 나머지는 참이에요.

거짓(False)으로 취급	참(True)으로 취급
0, 0.0	0이 아닌 숫자
"" (빈 문자열)	"a"
[], (), {} (빈 컬렉션)	[1], {"a": 1}
None	그 밖의 대부분

파이썬 코드
```
print(bool(0), bool(""), bool([]), bool(None))
print(bool(5), bool("hi"), bool([0]))
```

실행 결과
```
False False False False
True True True
```

04.6 자료형 돌아보기

자료형	기호	순서	변경	중복	예
리스트	[ ]	있음	가능	가능	[1, 2, 2]
튜플	( )	있음	불가	가능	(1, 2)
딕셔너리	{키: 값}	있음*	가능	키는 불가	{"a": 1}
집합	{ }	없음	가능	불가	{1, 2}
문자열	" "	있음	불가	가능	"abc"

*파이썬 3.7부터 딕셔너리는 넣은 순서를 기억해요.

파이썬 코드
```
# 상황에 맞는 자료형 고르기
shopping = ["우유", "빵", "우유"]          # 순서·중복 OK → 리스트
birthday = (2005, 3, 14)                  # 바뀌면 안 됨 → 튜플
phone_book = {"엄마": "010-1111-2222"}     # 이름으로 찾기 → 딕셔너리
visited = set(["서울", "부산", "서울"])     # 중복 없이 → 집합
print(shopping, birthday, phone_book, visited, sep="\n")
```

실행 결과

```
['우유', '빵', '우유']
(2005, 3, 14)
{'엄마': '010-1111-2222'}
{'부산', '서울'}
```

주의 빈 집합은 `set()` 으로 만들어요. `{}` 는 빈 **딕셔너리**입니다.

총정리 퀴즈

Q1. nums = [1, 2, 3] 에서 nums.append(4) 후 nums 는?

① [4, 1, 2, 3]
② [1, 2, 3, 4]
③ [1, 2, 3]
④ 오류

Q2. 튜플과 리스트의 가장 큰 차이는?

① 튜플은 숫자만 담는다
② 튜플은 만든 뒤 값을 바꿀 수 없다
③ 튜플은 순서가 없다
④ 튜플은 중복을 허용하지 않는다

Q3. d = {"a": 1} 에서 d.get("b", 0) 의 결과는?

① None
② 0
③ KeyError
④ "b"

Q4. set([1, 1, 2, 3, 3]) 의 원소 개수는?

① 5
② 3
③ 2
④ 4

Q5. bool("") 의 값은?

① True
② False
③ None
④ 오류

Q6. 빈 집합을 만드는 올바른 방법은?

① {}
② set()
③ []
④ ()

CHAPTER 05

흐름을 가르는 제어문, 조건문

학습 목표

- if, elif, else 로 상황에 따라 다른 코드를 실행할 수 있다.
- 들여쓰기가 왜 중요한지 안다.
- 비교 연산자와 논리 연산자(and, or, not)를 쓸 수 있다.
- try~except 로 오류가 나도 프로그램이 멈추지 않게 할 수 있다.

05.1 if 문

"비가 오면 우산을 챙긴다"처럼 **조건에 따라 할 일을 바꾸는 것**을 조건문이라고 해요.

파이썬 코드

```
temperature = 31
if temperature >= 30:
    print("너무 더워요!")       # 조건이 참일 때만 실행
    print("물을 많이 마셔요.")
print("오늘도 좋은 하루")         # if 와 상관없이 항상 실행
```

실행 결과

```
너무 더워요!
물을 많이 마셔요.
오늘도 좋은 하루
```

들여쓰기가 곧 문법

조건이 참일 때 실행할 코드는 **들여쓰기(보통 공백 4칸)**로 묶어요. 다른 언어의 중괄호 { } 역할을 파이썬은 들여쓰기가 합니다. if 줄 끝의 **콜론(:)**도 잊지 마세요.

파이썬 코드

```
if True:
print("들여쓰기를 안 하면?")
```

실행 결과

```
  File "<stdin>", line 2
    print("들여쓰기를 안 하면?")
IndentationError: expected an indented block after 'if' statement on line 1
```

else 와 elif

파이썬 코드

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:          # 위 조건이 거짓일 때 다음 조건 검사
    grade = "B"
elif score >= 70:
    grade = "C"
else:                      # 모든 조건이 거짓일 때
    grade = "F"
print(f"{score}점은 {grade}학점")
```

실행 결과

```
85점은 B학점
```

TIP elif 는 **위에서부터 차례로** 검사하고, 처음 참이 된 곳 하나만 실행해요. 그래서 조건의 순서가 중요합니다. (85점은 `>= 80` 에서 멈춰요)

짧은 조건식 (삼항 연산자)

파이썬 코드
```
age = 15
label = "성인" if age >= 19 else "청소년"
print(label)
```

실행 결과
```
청소년
```

05.2 비교 연산자, 논리 연산자

비교 연산자 — 결과는 True 또는 False

연산자	뜻	예 (a=5, b=3)
==	같다	a == b → False
!=	다르다	a != b → True
> , <	크다, 작다	a > b → True
>= , <=	크거나 같다, 작거나 같다	a <= 5 → True

주의 = 는 값을 넣는 대입, == 는 같은지 비교예요. if 에는 == 를 씁니다.

논리 연산자 — 조건 여러 개 합치기

연산자	뜻	예
and	둘 다 참이면 참	age >= 13 and age <= 18
or	하나라도 참이면 참	day == "토" or day == "일"
not	참·거짓 뒤집기	not is_member

파이썬 코드
```
age = 16
has_ticket = True
if age >= 13 and has_ticket:
    print("입장 가능")
day = "일"
if day == "토" or day == "일":
    print("주말이에요")
print(not has_ticket)
```

실행 결과
```
입장 가능
주말이에요
False
```

파이썬 코드

```
# 파이썬은 범위를 수학처럼 쓸 수 있다
n = 7
if 1 <= n <= 10:
    print("1부터 10 사이")
print(n % 2 == 0)          # 짝수인가?
```

실행 결과

```
1부터 10 사이
False
```

in 으로 포함 여부 검사

파이썬 코드

```
vip = ["김파이", "이코딩"]
name = "김파이"
if name in vip:
    print(f"{name}님, VIP 라운지로 오세요")
```

실행 결과

```
김파이님, VIP 라운지로 오세요
```

05.3 try~except 문

프로그램을 실행하다 오류가 나면 그 자리에서 멈춰 버려요. **try~except** 를 쓰면 오류가 나도 멈추지 않고 대신 할 일을 정할 수 있어요.

파이썬 코드

```
try:
    result = 10 / 0          # 0 으로 나누면 오류가 난다
    print(result)
except ZeroDivisionError:
    print("0 으로는 나눌 수 없어요.")
print("프로그램은 계속 실행돼요")
```

실행 결과

```
0 으로는 나눌 수 없어요.
프로그램은 계속 실행돼요
```

사용자가 숫자 대신 글자를 입력하는 경우도 자주 막아야 해요.

파이썬 코드

```
text = input("나이를 숫자로 입력하세요: ")
try:
    age = int(text)
    print(f"내년에는 {age + 1}살")
except ValueError:
    print("숫자만 입력해 주세요.")
```

실행 결과 · 입력값: 스물

```
나이를 숫자로 입력하세요: 스물
숫자만 입력해 주세요.
```

TIP 어떤 오류가 날지 알면 `except ValueError:` 처럼 오류 이름을 적는 게 좋아요. 오류 처리는 10장에서 더 깊게 배웁니다.

총정리 퀴즈

Q1. if 문에서 조건이 참일 때 실행할 코드를 묶는 방법은?

① 중괄호 { }
② 들여쓰기
③ 세미콜론 ;
④ end 로 닫기

Q2. score = 75 일 때 if score >= 80: A / elif score >= 70: B / else: C 의 결과는?

① A
② B
③ C
④ A 와 B

Q3. 같은지 비교하는 연산자는?

① =
② ==
③ ===
④ :=

Q4. True and False 의 결과는?

① True
② False
③ None
④ 오류

Q5. int("abc") 처럼 숫자로 바꿀 수 없을 때 나는 오류는?

① TypeError
② ValueError
③ NameError
④ KeyError

CHAPTER 06

반복을 만드는 제어문, 반복문

학습 목표

- while 문으로 조건이 참인 동안 반복할 수 있다.
- break 와 continue 로 반복을 멈추거나 건너뛸 수 있다.
- for 문과 range() 로 정해진 횟수만큼 반복할 수 있다.
- 리스트를 for 문으로 돌며 처리하고, 리스트 컴프리헨션을 쓸 수 있다.
- 중첩 반복문으로 구구단 같은 표를 만들 수 있다.

06.1 while 문과 반복문의 기초

같은 일을 여러 번 해야 할 때 코드를 복사해 붙이면 길고 고치기 어려워요. **반복문**을 쓰면 한 번만 쓰고 여러 번 실행할 수 있어요. **while 문**은 **조건이 참인 동안** 계속 반복합니다.

파이썬 코드

```
count = 1
while count <= 5:            # count 가 5 이하인 동안
    print(f"{count}번째 인사: 안녕!")
    count += 1               # 이 줄이 없으면 영원히 반복된다
print("반복 끝")
```

실행 결과

```
1번째 인사: 안녕!
2번째 인사: 안녕!
3번째 인사: 안녕!
4번째 인사: 안녕!
5번째 인사: 안녕!
반복 끝
```

주의 조건이 계속 참이면 **무한 반복**에 빠져요. 실행 중인 프로그램은 **Ctrl + C** 로 멈출 수 있어요. 반복할 때마다 조건이 바뀌는지 꼭 확인하세요.

break: 반복 멈추기, continue: 이번만 건너뛰기

파이썬 코드

```
n = 0
while True:                  # 일부러 무한 반복을 만들고
    n += 1
    if n % 2 == 0:
        continue             # 짝수면 아래를 건너뛰고 다시 처음으로
    if n > 9:
        break                # 9 를 넘으면 반복 탈출
    print(n, end=" ")
```

실행 결과

```
1 3 5 7 9
```

파이썬 코드

```
# 0 을 입력할 때까지 숫자를 더하는 계산기
total = 0
while True:
    num = int(input("숫자 (0=끝): "))
    if num == 0:
        break
    total += num
print("합계:", total)
```

실행 결과 · 입력값: 3, 7, 0

```
숫자 (0=끝): 3
숫자 (0=끝): 7
숫자 (0=끝): 0
합계: 10
```

06.2 for 문과 순회

for 문은 여러 값이 담긴 것에서 **하나씩 꺼내며** 반복해요. 하나씩 차례로 꺼내는 것을 **순회**라고 합니다.

파이썬 코드

```
for ch in "파이썬":
    print(ch)
```

실행 결과

```
파
이
썬
```

range(): 숫자 줄 만들기

쓰는 법	만들어지는 숫자
range(5)	0, 1, 2, 3, 4
range(1, 6)	1, 2, 3, 4, 5 (끝은 포함 안 함)
range(0, 10, 2)	0, 2, 4, 6, 8 (2씩 건너뛰기)
range(5, 0, -1)	5, 4, 3, 2, 1 (거꾸로)

파이썬 코드

```
for i in range(1, 6):
    print(i, end=" ")
print()
total = 0
for i in range(1, 101):
    total += i
print("1부터 100까지의 합:", total)
```

실행 결과

```
1 2 3 4 5
1부터 100까지의 합: 5050
```

TIP 횟수가 정해져 있으면 **for**, 언제 끝날지 모르고 조건으로 멈추면 **while** 을 쓰는 게 자연스러워요.

06.3 리스트와 반복문

파이썬 코드

```
scores = [88, 72, 95, 60]
total = 0
for s in scores:
    total += s
print("평균:", total / len(scores))
```

실행 결과

```
평균: 78.75
```

enumerate(): 번호와 값 함께, zip(): 두 리스트 나란히

파이썬 코드

```
names = ["김파이", "이코딩", "박자바"]
for i, name in enumerate(names, start=1):
    print(f"{i}번: {name}")
scores = [90, 85, 77]
for name, score in zip(names, scores):
    print(name, score)
```

실행 결과

```
1번: 김파이
2번: 이코딩
3번: 박자바
김파이 90
이코딩 85
박자바 77
```

조건에 맞는 값만 모으기

파이썬 코드

```
scores = [88, 72, 95, 60]
passed = []
for s in scores:
    if s >= 80:
        passed.append(s)
print(passed)
```

실행 결과

```
[88, 95]
```

리스트 컴프리헨션: 한 줄로 리스트 만들기

위 코드를 한 줄로 쓸 수 있어요. `[식 for 변수 in 반복할것 if 조건]` 형태입니다.

파이썬 코드

```
scores = [88, 72, 95, 60]
passed = [s for s in scores if s >= 80]
squares = [n ** 2 for n in range(1, 6)]
print(passed, squares)
```

실행 결과

```
[88, 95] [1, 4, 9, 16, 25]
```

딕셔너리 돌기

파이썬 코드

```
prices = {"커피": 4500, "라떼": 5000, "케이크": 6500}
for menu, price in prices.items():
    print(f"{menu}: {price:,}원")      # :, → 천 단위 쉼표
```

실행 결과

```
커피: 4,500원
라떼: 5,000원
케이크: 6,500원
```

06.4 중첩 반복문

반복문 안에 반복문을 넣을 수 있어요. 바깥 반복이 한 번 돌 때 안쪽 반복은 처음부터 끝까지 다 돕니다.

파이썬 코드

```
for dan in range(2, 4):            # 2단, 3단
    for i in range(1, 10):         # 1 ~ 9
        print(f"{dan}×{i}={dan * i:2}", end="  ")
    print()                        # 한 단이 끝나면 줄바꿈
```

실행 결과

```
2×1= 2  2×2= 4  2×3= 6  2×4= 8  2×5=10  2×6=12  2×7=14  2×8=16  2×9=18
3×1= 3  3×2= 6  3×3= 9  3×4=12  3×5=15  3×6=18  3×7=21  3×8=24  3×9=27
```

파이썬 코드

```
# 별로 삼각형 그리기
for row in range(1, 6):
    print("*" * row)
```

실행 결과

```
*
**
***
****
*****
```

파이썬 코드

```
# 2차원 리스트(표) 돌기
table = [[1, 2, 3], [4, 5, 6]]
for row in table:
    for value in row:
        print(value, end=" ")
    print()
```

실행 결과

```
1 2 3
4 5 6
```

총정리 퀴즈

Q1. while 문은 언제까지 반복하나요?

① 정해진 횟수만큼
② 조건이 참인 동안
③ 리스트가 끝날 때까지
④ 한 번만

Q2. range(1, 5) 가 만드는 숫자는?

① 1, 2, 3, 4, 5
② 1, 2, 3, 4
③ 0, 1, 2, 3, 4
④ 2, 3, 4, 5

Q3. 반복문을 즉시 끝내는 명령은?

① continue
② break
③ stop
④ exit

Q4. [n * 2 for n in range(3)] 의 결과는?

① [2, 4, 6]
② [0, 2, 4]
③ [0, 1, 2]
④ [1, 2, 3]

Q5. 바깥 for 가 3번, 안쪽 for 가 4번 돌면 안쪽 코드는 몇 번 실행되나요?

① 7번
② 12번
③ 4번
④ 3번

CHAPTER 07

코드를 묶어 재사용할 수 있는 함수

학습 목표

- 함수를 정의(def)하고 호출할 수 있다.
- 매개변수와 인수, return 으로 값을 돌려주는 법을 안다.
- 기본값·키워드 인수·가변 인수를 쓸 수 있다.
- lambda 로 짧은 함수를 만들 수 있다.
- 지역 변수와 전역 변수(스코프)를 구분한다.
- 재귀 함수의 원리를 이해하고, 자주 쓰는 내장 함수를 안다.

07.1 함수의 개념 알아보기

함수는 자주 쓰는 코드를 **이름 붙여 묶어 둔 것**이에요. 한 번 만들어 두면 이름만 불러서(호출) 몇 번이고 쓸 수 있어요. `print()`, `len()` 도 파이썬이 미리 만들어 둔 함수입니다.

파이썬 코드

```
def say_hello():            # def 함수이름(): 으로 정의
    print("안녕하세요!")
    print("파이썬 함수 시간입니다.")

say_hello()                 # 호출: 이름 뒤에 () 를 붙인다
say_hello()
```

실행 결과

```
안녕하세요!
파이썬 함수 시간입니다.
안녕하세요!
파이썬 함수 시간입니다.
```

TIP 함수를 쓰면 ① 같은 코드를 반복해서 쓰지 않아도 되고 ② 고칠 때 한 곳만 고치면 되고 ③ 이름만 봐도 무슨 일을 하는지 알 수 있어요.

매개변수: 함수에 값 넘겨주기

파이썬 코드

```
def greet(name):            # name 은 매개변수 (값을 받는 자리)
    print(f"{name}님, 반가워요!")

greet("김파이")              # "김파이" 는 인수 (넘겨주는 값)
greet("이코딩")
```

실행 결과

```
김파이님, 반가워요!
이코딩님, 반가워요!
```

07.2 함수와 반환 알아보기

`return` 은 함수의 계산 결과를 **호출한 곳으로 돌려줘요.** 돌려받은 값은 변수에 담거나 다른 계산에 쓸 수 있습니다.

파이썬 코드
```
def add(a, b):
    return a + b            # 결과를 돌려주고 함수를 끝낸다

result = add(3, 4)
print(result)
print(add(10, 20) * 2)      # 반환값을 바로 계산에 쓰기
```

실행 결과
```
7
60
```

print 와 return 은 달라요

파이썬 코드
```
def show_double(x):
    print(x * 2)            # 화면에 보여 주기만 한다

def get_double(x):
    return x * 2            # 값을 돌려준다

a = show_double(5)          # 10 이 출력되지만
b = get_double(5)
print("a =", a)             # a 에는 아무 값도 없다 (None)
print("b =", b)
```

실행 결과
```
10
a = None
b = 10
```

값을 여러 개 돌려주기

파이썬 코드
```
def min_max(numbers):
    return min(numbers), max(numbers)     # 튜플로 돌려준다

low, high = min_max([7, 2, 9, 4])
print(f"최솟값 {low}, 최댓값 {high}")
```

실행 결과
```
최솟값 2, 최댓값 9
```

TIP return 을 만나면 함수는 **그 자리에서 끝나요.** 그 아래 코드는 실행되지 않습니다.

07.3 매개변수 좀 더 깊게 알아보기

기본값과 키워드 인수

파이썬 코드
```
def order(menu, size="보통", count=1):    # 기본값이 있는 매개변수는 뒤에
    print(f"{menu} {size} {count}개 주문")

order("라면")                             # 기본값 사용
order("짜장면", "곱빼기")
order("커피", count=3)                    # 이름=값 으로 원하는 것만 지정
```

실행 결과
```
라면 보통 1개 주문
짜장면 곱빼기 1개 주문
커피 보통 3개 주문
```

가변 인수: *args (개수 제한 없이), **kwargs (이름=값 여러 개)

파이썬 코드
```
def total(*nums):          # 넘긴 값들이 튜플로 모인다
    return sum(nums)

print(total(1, 2), total(1, 2, 3, 4, 5))

def profile(**info):       # 이름=값 들이 딕셔너리로 모인다
    for key, value in info.items():
        print(f"{key}: {value}")

profile(name="김파이", age=20, city="서울")
```

실행 결과
```
3 15
name: 김파이
age: 20
city: 서울
```

주의 기본값으로 리스트 같은 바뀌는 값을 쓰면(`def f(a=[])`) 호출할 때마다 같은 리스트가 공유되는 함정이 있어요. 기본값은 `None` 으로 두고 함수 안에서 새로 만드세요.

07.4 함수를 한 줄로 정의할 수 있는 람다

`lambda 매개변수: 식` 은 이름 없는 짧은 함수예요. 주로 다른 함수에 **잠깐 넘겨줄 때** 씁니다.

파이썬 코드
```
square = lambda x: x ** 2
print(square(5))

students = [("김파이", 85), ("이코딩", 92), ("박자바", 78)]
students.sort(key=lambda s: s[1], reverse=True)    # 점수(1번 값)로 정렬
print(students)
```

실행 결과
```
25
[('이코딩', 92), ('김파이', 85), ('박자바', 78)]
```

파이썬 코드
```
nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * 10, nums)))        # 모두에 적용
print(list(filter(lambda n: n % 2 == 0, nums))) # 조건에 맞는 것만
```

실행 결과
```
[10, 20, 30, 40, 50, 60]
[2, 4, 6]
```

07.5 함수와 스코프

변수가 **쓰일 수 있는 범위**를 스코프라고 해요.

- **지역 변수**: 함수 안에서 만든 변수. 함수가 끝나면 사라지고, 밖에서는 보이지 않아요.
- **전역 변수**: 함수 밖에서 만든 변수. 어디서든 읽을 수 있어요.

파이썬 코드
```
message = "전역"            # 전역 변수

def func():
    message = "지역"        # 같은 이름이지만 함수 안의 새 변수
    print("함수 안:", message)

func()
print("함수 밖:", message)   # 전역 변수는 그대로
```

실행 결과
```
함수 안: 지역
함수 밖: 전역
```

파이썬 코드
```
def make_number():
    secret = 42
make_number()
print(secret)               # 지역 변수는 밖에서 쓸 수 없다
```

실행 결과
```
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    print(secret)               # 지역 변수는 밖에서 쓸 수 없다
NameError: name 'secret' is not defined
```

global 로 전역 변수 바꾸기

파이썬 코드
```
count = 0
def increase():
    global count            # 전역 변수 count 를 쓰겠다는 선언
    count += 1
increase(); increase()
print(count)
```

실행 결과
```
2
```

TIP `global` 은 꼭 필요할 때만 쓰세요. 함수는 **매개변수로 받고 return 으로 돌려주는 것**이 가장 깔끔합니다.

07.6 재귀 함수 알아보기

재귀 함수는 자기 자신을 다시 부르는 함수예요. 큰 문제를 같은 모양의 작은 문제로 나눠 풀 때 씁니다. 반드시 **멈추는 조건(종료 조건)**이 있어야 해요.

파이썬 코드

```
def countdown(n):
    if n == 0:              # 종료 조건
        print("발사!")
        return
    print(n)
    countdown(n - 1)        # 더 작은 문제로 자기 자신 호출

countdown(3)
```

실행 결과

```
3
2
1
발사!
```

파이썬 코드

```
def factorial(n):           # n! = n × (n-1)!
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))         # 5 × 4 × 3 × 2 × 1
```

실행 결과

```
120
```

주의 종료 조건이 없으면 끝없이 자신을 불러 `RecursionError` 가 나요. 파이썬은 기본적으로 약 1000번까지만 재귀를 허용합니다.

07.7 알아두면 좋은 파이썬 내장 함수 6가지

함수	하는 일	예	결과
abs(x)	절댓값	abs(-7)	7
round(x, n)	반올림	round(3.146, 2)	3.15
sum(목록)	합계	sum([1, 2, 3])	6
max / min	최댓값 / 최솟값	max(3, 9, 4)	9
sorted(목록)	정렬된 새 리스트	sorted([3, 1, 2])	[1, 2, 3]
enumerate / zip	번호 붙이기 / 나란히 묶기	list(zip("ab", [1, 2]))	[('a', 1), ('b', 2)]

파이썬 코드

```
data = [-3, 8, 1, -9, 5]
print(abs(-3), round(2.675, 1), sum(data))
print(max(data), min(data), max(data, key=abs))   # key: 기준을 정할 수 있다
print(sorted(data, reverse=True))
for i, (a, b) in enumerate(zip("가나", "AB")):
    print(i, a, b)
```

실행 결과

```
3 2.7 2
8 -9 -9
[8, 5, 1, -3, -9]
0 가 A
1 나 B
```

TIP 더 많은 내장 함수는 **부록 B** 에 정리해 두었어요.

총정리 퀴즈

Q1. 함수를 정의할 때 쓰는 키워드는?

① function
② def
③ func
④ define

Q2. def f(x): return x * 3 일 때 f(4) 의 결과는?

① 7
② 12
③ 43
④ None

Q3. return 이 없는 함수를 호출한 결과를 변수에 담으면 그 값은?

① 0
② None
③ 빈 문자열
④ 오류

Q4. def order(menu, count=1) 에서 order("밥") 을 호출하면 count 는?

① 0
② 1
③ None
④ 오류

Q5. (lambda x: x + 1)(5) 의 결과는?

① 5
② 6
③ x+1
④ 오류

Q6. 재귀 함수에 반드시 있어야 하는 것은?

① for 문
② 종료 조건
③ global 변수
④ lambda

CHAPTER 08

자료와 함수를 묶어 관리하는 클래스

학습 목표

- 클래스와 인스턴스(객체)의 관계를 설명할 수 있다.
- __init__ 과 self 로 객체의 속성을 만들 수 있다.
- 상속으로 기존 클래스를 넓히고, 메서드 오버라이딩으로 다형성을 만들 수 있다.
- 클래스 변수와 인스턴스 변수, 메서드 종류를 구분한다.

08.1 클래스와 인스턴스

게임 속 캐릭터를 생각해 보세요. 캐릭터마다 이름·체력(**데이터**)이 있고, 공격하기·회복하기(**기능**)를 할 수 있어요. **클래스**는 이렇게 데이터와 기능을 하나로 묶은 **설계도**이고, 설계도로 만든 실제 물건을 **인스턴스(객체)**라고 합니다.

비유	클래스	인스턴스
붕어빵	붕어빵 틀	하나하나의 붕어빵
게임	캐릭터 설계	전사 "아서", 마법사 "멀린"

파이썬 코드

```
class Character:                    # 클래스 이름은 대문자로 시작 (PascalCase)
    def __init__(self, name, hp):    # 인스턴스를 만들 때 자동으로 실행
        self.name = name             # self.이름 = 인스턴스마다 따로 가지는 값(속성)
        self.hp = hp

    def attack(self, target):        # 메서드: 클래스 안의 함수
        print(f"{self.name}이(가) {target.name}을(를) 공격!")
        target.hp -= 10

hero = Character("아서", 100)         # 인스턴스 만들기
monster = Character("슬라임", 30)
hero.attack(monster)
print(monster.name, "남은 체력:", monster.hp)
```

실행 결과

```
아서이(가) 슬라임을(를) 공격!
슬라임 남은 체력: 20
```

TIP **self** 는 "지금 이 인스턴스 자신"을 가리켜요. 메서드의 첫 매개변수로 꼭 쓰고, 부를 때는 `hero.attack(monster)` 처럼 self 자리를 비워 둡니다. 파이썬이 알아서 hero 를 넣어 줘요.

인스턴스는 서로 독립적이에요

파이썬 코드

```
class Counter:
    def __init__(self):
        self.value = 0
    def up(self):
        self.value += 1

a = Counter(); b = Counter()
a.up(); a.up(); b.up()
print(a.value, b.value)
```

실행 결과

```
2 1
```

08.2 상속과 다형성

상속은 이미 있는 클래스(부모)의 속성과 메서드를 그대로 물려받아 새 클래스(자식)를 만드는 것이에요. 공통 부분은 부모에, 다른 부분만 자식에 쓰면 됩니다.

파이썬 코드

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name}: ...")

class Dog(Animal):                  # Animal 을 물려받는다
    def speak(self):                # 같은 이름으로 다시 정의 (오버라이딩)
        print(f"{self.name}: 멍멍!")

class Cat(Animal):
    def speak(self):
        print(f"{self.name}: 야옹~")

for pet in [Dog("바둑이"), Cat("나비"), Animal("거북이")]:
    pet.speak()                     # 같은 명령, 다른 동작 → 다형성
```

실행 결과

```
바둑이: 멍멍!
나비: 야옹~
거북이: ...
```

다형성은 같은 이름의 메서드를 불러도 객체의 종류에 따라 다르게 동작하는 것이에요. 위에서 `speak()` 하나로 개·고양이가 각자 다르게 울죠.

super(): 부모의 기능을 이어서 쓰기

파이썬 코드

```
class Student:
    def __init__(self, name):
        self.name = name

class Graduate(Student):
    def __init__(self, name, thesis):
        super().__init__(name)       # 부모의 __init__ 으로 name 설정
        self.thesis = thesis         # 자식만의 속성 추가

g = Graduate("김파이", "AI 연구")
print(g.name, "/", g.thesis)
print(isinstance(g, Student))        # 자식은 부모의 한 종류
```

실행 결과

```
김파이 / AI 연구
True
```

08.3 클래스 변수와 메서드

구분	만드는 곳	공유	예
인스턴스 변수	__init__ 안 self.이름	인스턴스마다 따로	self.name
클래스 변수	클래스 바로 아래	모든 인스턴스가 공유	count = 0

파이썬 코드

```
class Member:
    count = 0                        # 클래스 변수: 모든 회원이 함께 쓰는 값

    def __init__(self, name):
        self.name = name             # 인스턴스 변수
        Member.count += 1            # 회원이 생길 때마다 1 증가

Member("김파이"); Member("이코딩"); m = Member("박자바")
print("전체 회원 수:", Member.count, "/", m.count)
```

실행 결과

```
전체 회원 수: 3 / 3
```

메서드의 종류

파이썬 코드

```
class MathTool:
    pi = 3.14159

    def __init__(self, r):
        self.r = r
    def area(self):                 # 인스턴스 메서드: self 사용
        return MathTool.pi * self.r ** 2

    @classmethod
    def info(cls):                  # 클래스 메서드: 클래스(cls) 사용
        return f"원주율 {cls.pi}"

    @staticmethod
    def add(a, b):                  # 정적 메서드: self·cls 없이 독립적인 기능
        return a + b

print(round(MathTool(2).area(), 2), MathTool.info(), MathTool.add(3, 4))
```

실행 결과

```
12.57 원주율 3.14159 7
```

__str__: print 했을 때 보기 좋게

파이썬 코드

```
class Book:
    def __init__(self, title, price):
        self.title, self.price = title, price
    def __str__(self):
        return f"『 {self.title} 』 {self.price:,}원"

print(Book("파이썬 첫걸음", 18000))
```

실행 결과

```
『 파이썬 첫걸음 』 18,000원
```

08.4 클래스 연습하기

은행 계좌 클래스를 만들어 봐요. 입금·출금 기능과, 잔액이 부족할 때의 처리까지 넣습니다.

파이썬 코드

```
class BankAccount:
    bank = "파이은행"                   # 클래스 변수

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = []               # 거래 기록

    def deposit(self, amount):
        self.balance += amount
        self.history.append(f"입금 {amount:,}")

    def withdraw(self, amount):
        if amount > self.balance:
            print(f"잔액 부족! (잔액 {self.balance:,}원)")
            return False
        self.balance -= amount
```

```
        self.history.append(f"출금 {amount:,}")
        return True

    def __str__(self):
        return f"[{self.bank}] {self.owner} 님 잔액 {self.balance:,}원"

acc = BankAccount("김파이", 10000)
acc.deposit(5000)
acc.withdraw(30000)
acc.withdraw(3000)
print(acc)
print(acc.history)
```

실행 결과
```
잔액 부족! (잔액 15,000원)
[파이은행] 김파이 님 잔액 12,000원
['입금 5,000', '출금 3,000']
```

파이썬 코드
```
# 상속으로 이자 기능이 있는 적금 계좌 만들기
class SavingAccount(BankAccount):
    def __init__(self, owner, balance=0, rate=0.03):
        super().__init__(owner, balance)
        self.rate = rate
    def add_interest(self):
        interest = int(self.balance * self.rate)
        self.deposit(interest)
        print(f"이자 {interest:,}원 지급")

s = SavingAccount("이코딩", 100000)
s.add_interest()
print(s)
```

실행 결과
```
이자 3,000원 지급
[파이은행] 이코딩 님 잔액 103,000원
```

총정리 퀴즈

Q1. 클래스로 만든 실제 객체를 무엇이라 하나요?

① 메서드
② 인스턴스
③ 모듈
④ 매개변수

Q2. 인스턴스를 만들 때 자동으로 실행되는 메서드는?

① __str__
② __init__
③ __main__
④ start

Q3. 메서드의 첫 번째 매개변수 self 가 가리키는 것은?

① 클래스 이름
② 메서드를 부른 인스턴스 자신
③ 부모 클래스
④ 전역 변수

Q4. class Dog(Animal): 에서 Animal 은 Dog 의 무엇인가요?

① 인스턴스
② 부모 클래스
③ 메서드
④ 클래스 변수

Q5. 모든 인스턴스가 함께 공유하는 변수는?

① 지역 변수
② 인스턴스 변수
③ 클래스 변수
④ 매개변수

CHAPTER 09

입력과 출력

학습 목표

- print() 의 출력 방식과 문자열 포맷팅을 자유롭게 쓴다.
- 파일을 만들고 글을 쓰거나(w) 덧붙일(a) 수 있다.
- input() 으로 키보드 입력을 받고 자료형을 바꿔 쓸 수 있다.
- 파일을 읽고(r), 한 줄씩 처리할 수 있다.
- 입출력을 모아 간단한 메모장 프로그램을 만든다.

09.1 출력이란 무엇일까?

출력은 프로그램이 결과를 바깥으로 내보내는 것이에요. 화면에 보여 주기(`print`), 파일에 저장하기, 인터넷으로 보내기 모두 출력입니다. 2장에서 배운 print 기능을 정리하고, 표처럼 줄 맞춰 출력하는 법을 더 배워 봐요.

파이썬 코드

```
name, score = "김파이", 92.456
print("이름:", name, "점수:", score)           # 쉼표: 띄어쓰기로 연결
print("이름: " + name)                          # +: 문자열끼리만
print(f"이름: {name}, 점수: {score:.1f}")        # f-문자열 (가장 추천)
print("이름: {}, 점수: {:.1f}".format(name, score))  # format() 메서드
```

실행 결과

```
이름: 김파이 점수: 92.456
이름: 김파이
이름: 김파이, 점수: 92.5
이름: 김파이, 점수: 92.5
```

자릿수 맞추기

형식	뜻	예	결과
{x:10}	10칸 (글자는 왼쪽 정렬)	f"[{'abc':5}]"	[abc]
{x:>10}	10칸 오른쪽 정렬	f"[{'abc':>5}]"	[abc]
{x:^10}	10칸 가운데 정렬	f"[{'abc':^5}]"	[abc]
{x:.2f}	소수점 둘째 자리	f"{3.14159:.2f}"	3.14
{x:,}	천 단위 쉼표	f"{1234567:,}"	1,234,567
{x:05}	빈자리를 0으로	f"{42:05}"	00042

파이썬 코드

```
items = [("커피", 4500), ("샌드위치", 7800), ("쿠키", 2000)]
print(f"{'메뉴':^8}¦{'가격':>8}")
print("-" * 18)
for menu, price in items:
    print(f"{menu:<8}¦{price:>8,}")
```

실행 결과

```
   메뉴   ┆      가격
------------------
커피      ┆   4,500
샌드위치    ┆   7,800
쿠키      ┆   2,000
```

09.2 파일 출력 이해하기

프로그램이 끝나면 변수의 값은 사라져요. 오래 남기려면 **파일**에 저장해야 합니다. `open(파일이름, 모드)` 로 파일을 열고, 다 쓰면 닫아요.

모드	뜻	파일이 없으면	파일이 있으면
"w"	쓰기	새로 만든다	**내용을 지우고** 새로 쓴다
"a"	덧붙이기	새로 만든다	끝에 이어 쓴다
"r"	읽기 (기본값)	오류	읽는다

파이썬 코드

```
f = open("hello.txt", "w", encoding="utf-8")   # 한글은 encoding="utf-8" 을 꼭!
f.write("안녕하세요\n")                          # write 는 줄바꿈을 자동으로 넣지 않는다
f.write("파일에 저장된 두 번째 줄\n")
f.close()                                      # 다 쓰면 닫기
print("hello.txt 저장 완료")
```

실행 결과

```
hello.txt 저장 완료
```

with 문: 자동으로 닫아 주는 안전한 방법

파이썬 코드

```
with open("diary.txt", "w", encoding="utf-8") as f:
    f.write("11월 4일: 파이썬 파일 저장을 배웠다.\n")
with open("diary.txt", "a", encoding="utf-8") as f:   # a: 이어 쓰기
    f.write("11월 5일: 파일 읽기를 배울 예정.\n")
    print("print 로도 파일에 쓸 수 있다", file=f)
print("diary.txt 에 3줄 저장")
```

실행 결과

```
diary.txt 에 3줄 저장
```

TIP with 블록이 끝나면 파일이 **자동으로 닫혀요.** close() 를 잊을 걱정이 없으니 항상 with 를 쓰는 습관을 들이세요.

09.3 입력이란 무엇일까?

입력은 바깥에서 프로그램으로 값을 받아들이는 것이에요. 키보드, 파일, 센서, 인터넷 모두 입력이 될 수 있어요. 가장 기본은 키보드로 입력받는 `input()` 입니다.

파이썬 코드

```
name = input("이름을 입력하세요: ")     # 안내 문구를 보여 주고 Enter 까지 기다린다
print(f"{name}님, 환영합니다!")
```

실행 결과 · 입력값: 김파이
```
이름을 입력하세요: 김파이
김파이님, 환영합니다!
```

input() 은 항상 문자열을 돌려줘요

파이썬 코드
```
a = input("첫 번째 숫자: ")
b = input("두 번째 숫자: ")
print("그냥 더하면:", a + b)             # 문자열끼리 붙어 버린다
print("숫자로 바꿔 더하면:", int(a) + int(b))
```

실행 결과 · 입력값: 20, 30
```
첫 번째 숫자: 20
두 번째 숫자: 30
그냥 더하면: 2030
숫자로 바꿔 더하면: 50
```

한 줄에 여러 값 입력받기

파이썬 코드
```
nums = input("숫자 여러 개 (띄어쓰기로): ").split()   # ['3', '5', '8']
nums = [int(n) for n in nums]                       # 모두 정수로
print("합계:", sum(nums), "평균:", sum(nums) / len(nums))
```

실행 결과 · 입력값: 3 5 8
```
숫자 여러 개 (띄어쓰기로): 3 5 8
합계: 16 평균: 5.333333333333333
```

TIP `map(int, input().split())` 처럼 써도 같은 결과예요. 코딩 테스트에서 자주 보는 형태입니다.

09.4 파일 입력 이해하기

앞에서 저장한 `diary.txt` 를 읽어 봐요.

파이썬 코드
```
with open("diary.txt", "r", encoding="utf-8") as f:
    content = f.read()                  # 전체를 문자열 하나로
print(content)
```

실행 결과
```
11월 4일: 파이썬 파일 저장을 배웠다.
11월 5일: 파일 읽기를 배울 예정.
print 로도 파일에 쓸 수 있다
```

메서드	돌려주는 값
read()	파일 전체를 문자열 하나로
readline()	한 줄만 (부를 때마다 다음 줄)
readlines()	모든 줄을 리스트로 ['첫 줄₩n', '둘째 줄₩n']

파이썬 코드
```
with open("diary.txt", encoding="utf-8") as f:   # 모드를 생략하면 "r"
    lines = f.readlines()
print(len(lines), "줄")
print(lines[0].strip())                          # strip() 으로 줄 끝의 \n 제거
```

실행 결과
```
3 줄
11월 4일: 파이썬 파일 저장을 배웠다.
```

주의 없는 파일을 "r" 로 열면 `FileNotFoundError` 가 나요. 파일 이름과 위치(폴더)를 확인하세요. 10장에서 이 오류를 처리하는 법을 배워요.

09.5 파일을 한 줄씩 읽어 처리하기

파일 객체는 for 문으로 돌면 **한 줄씩** 꺼내 줘요. 큰 파일도 메모리를 적게 쓰며 처리할 수 있어요.

파이썬 코드
```
# 성적 파일 만들기 (이름,점수)
with open("scores.csv", "w", encoding="utf-8") as f:
    f.write("김파이,88\n이코딩,95\n박자바,72\n최루비,64\n")

total, count = 0, 0
with open("scores.csv", encoding="utf-8") as f:
    for line in f:                          # 한 줄씩
        name, score = line.strip().split(",")
        score = int(score)
        mark = "합격" if score >= 70 else "불합격"
        print(f"{name}: {score}점 {mark}")
        total += score; count += 1
print(f"평균 {total / count:.1f}점")
```

실행 결과
```
김파이: 88점 합격
이코딩: 95점 합격
박자바: 72점 합격
최루비: 64점 불합격
평균 79.8점
```

파이썬 코드
```
# 합격자만 골라 새 파일에 저장하기
with open("scores.csv", encoding="utf-8") as src, \
     open("passed.txt", "w", encoding="utf-8") as dst:
    for line in src:
        name, score = line.strip().split(",")
        if int(score) >= 70:
            dst.write(name + "\n")
print(open("passed.txt", encoding="utf-8").read())
```

실행 결과
```
김파이
이코딩
박자바
```

09.6 메모장 프로그램 만들어보기

지금까지 배운 입력·출력·파일·반복·조건을 모아 **메모장 프로그램**을 만들어 봐요. 메모를 쓰면 `memo.txt` 에 저장되고, 목록 보기로 저장된 메모를 볼 수 있어요.

파이썬 코드

```
# memo.py — 메모장 프로그램 (1: 쓰기, 2: 목록, 3: 끝)
FILE = "memo.txt"
open(FILE, "w", encoding="utf-8").close()     # 실습을 위해 빈 파일로 시작

while True:
    menu = input("\n[1] 메모 쓰기 [2] 목록 보기 [3] 끝내기 > ")
    if menu == "1":
        text = input("메모 내용: ")
        with open(FILE, "a", encoding="utf-8") as f:
            f.write(text + "\n")
        print("저장했어요.")
    elif menu == "2":
        with open(FILE, encoding="utf-8") as f:
            for i, line in enumerate(f, start=1):
                print(f"{i}. {line.strip()}")
    elif menu == "3":
        print("메모장을 종료합니다.")
        break
    else:
        print("1, 2, 3 중에서 골라 주세요.")
```

실행 결과 · 입력값: 1, 우유 사기, 1, 파이썬 복습, 2, 3

```
[1] 메모 쓰기 [2] 목록 보기 [3] 끝내기 > 1
메모 내용: 우유 사기
저장했어요.

[1] 메모 쓰기 [2] 목록 보기 [3] 끝내기 > 1
메모 내용: 파이썬 복습
저장했어요.

[1] 메모 쓰기 [2] 목록 보기 [3] 끝내기 > 2
1. 우유 사기
2. 파이썬 복습

[1] 메모 쓰기 [2] 목록 보기 [3] 끝내기 > 3
메모장을 종료합니다.
```

TIP 이 프로그램은 메모 파일이 없거나 이상한 입력이 들어오면 문제가 생길 수 있어요. 10장에서 **오류에 대비한 메모장**으로 업그레이드합니다.

총정리 퀴즈

Q1. f"{3.14159:.2f}" 의 결과는?

① 3.1
② 3.14
③ 3.142
④ 3.14159

Q2. 파일 모드 "w" 로 이미 있는 파일을 열면?

① 끝에 이어 쓴다
② 기존 내용을 지우고 새로 쓴다
③ 오류가 난다
④ 읽기만 한다

Q3. with open(...) as f: 를 쓰는 가장 큰 이유는?

① 속도가 빨라서
② 블록이 끝나면 파일을 자동으로 닫아 줘서
③ 한글을 쓰려고
④ 파일을 숨기려고

Q4. input() 으로 받은 "5" 와 "3" 을 + 하면?

① 8
② "53"
③ 오류
④ 15

Q5. 파일의 모든 줄을 리스트로 읽는 메서드는?

① read()
② readline()
③ readlines()
④ lines()

CHAPTER 10

오류 읽고 예외 처리하기

학습 목표

- 오류 메시지(Traceback)를 읽고 원인을 찾을 수 있다.
- 자주 만나는 오류의 종류와 뜻을 안다.
- 필요한 예외만 골라 처리하고, as 로 오류 내용을 받을 수 있다.
- else 와 finally 의 실행 시점을 안다.
- raise 로 직접 예외를 일으키고, 오류에 강한 프로그램을 만든다.

10.1 오류는 어떻게 읽고, 어떻게 처리해야 할까?

오류(에러)는 실패가 아니라 **컴퓨터가 알려 주는 힌트**예요. 오류 메시지를 읽는 법만 알면 대부분 스스로 고칠 수 있어요.

파이썬 코드

```
prices = [1000, 2000, 3000]
print(prices[3])
```

실행 결과

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    print(prices[3])
IndexError: list index out of range
```

Traceback 읽는 순서

1. **맨 아래 줄부터** 읽어요. `IndexError: list index out of range` → 오류 종류와 이유
2. 그 위의 `line 2` → 몇 번째 줄에서 났는지
3. 그 줄의 코드를 보고 원인을 찾아요. (리스트는 0~2번까지인데 3번을 꺼냈어요)

자주 만나는 오류

오류	뜻	흔한 원인
SyntaxError	문법이 틀림	콜론·괄호·따옴표 빠짐
IndentationError	들여쓰기가 틀림	공백 개수가 다름
NameError	모르는 이름	오타, 변수를 만들기 전에 사용
TypeError	자료형이 맞지 않음	"a" + 1
ValueError	값이 알맞지 않음	int("abc")
IndexError	없는 번호	리스트 범위를 넘음
KeyError	없는 키	딕셔너리에 없는 키
ZeroDivisionError	0으로 나눔	10 / 0
FileNotFoundError	파일이 없음	파일 이름·경로 오타

TIP **문법 오류**(SyntaxError)는 실행 전에 발견되어 아예 실행되지 않고, 나머지는 실행 중에 생기는 **예외(exception)**예요. 예외는 try~except 로 처리할 수 있어요.

10.2 필요한 예외만 골라 처리하기

`except:` 만 쓰면 모든 오류를 다 잡아 버려서, 진짜 원인을 숨길 수 있어요. **예상되는 오류만 이름을 적어** 처리하는 것이 좋습니다.

파이썬 코드

```
try:
    a = int(input("나눌 수: "))
    b = int(input("나누는 수: "))
    print("결과:", a / b)
except ValueError:
    print("숫자를 입력해 주세요.")
except ZeroDivisionError:
    print("0 으로 나눌 수 없어요.")
```

실행 결과 · 입력값: 10, 0

```
나눌 수: 10
나누는 수: 0
0 으로 나눌 수 없어요.
```

여러 오류를 한 번에, as 로 오류 내용 받기

파이썬 코드

```
data = {"a": 1}
for key in ["a", "b"]:
    try:
        print(key, "→", 10 / (data[key] - 1))
    except (KeyError, ZeroDivisionError) as e:   # 괄호로 묶어 여러 개
        print(f"{key}: {type(e).__name__} 발생 ({e})")
```

실행 결과

```
a: ZeroDivisionError 발생 (division by zero)
b: KeyError 발생 ('b')
```

주의 `except Exception:` 처럼 모든 예외를 잡을 때는 최소한 오류 내용을 출력하거나 기록하세요. 아무것도 안 하고 넘기면(`pass`) 문제를 찾기 아주 어려워져요.

10.3 else 와 finally 로 실행 흐름 나누기

블록	실행되는 때
try	먼저 시도
except	try 에서 오류가 났을 때
else	try 에서 오류가 **없었을 때**
finally	오류가 있든 없든 **항상** 마지막에

파이썬 코드

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("  → 0으로 나눌 수 없음")
    else:
        print("  → 성공:", result)
    finally:
        print("  → 계산 끝 (항상 실행)")

print("10 / 2"); divide(10, 2)
print("10 / 0"); divide(10, 0)
```

실행 결과

```
10 / 2
  → 성공: 5.0
  → 계산 끝 (항상 실행)
10 / 0
  → 0으로 나눌 수 없음
  → 계산 끝 (항상 실행)
```

TIP finally 는 파일 닫기, 연결 끊기처럼 **무슨 일이 있어도 꼭 해야 하는 정리 작업**에 써요.

10.4 raise 로 직접 예외 발생시키기

문법적으로는 문제없지만 **규칙에 맞지 않는 값**일 때, 직접 예외를 일으킬 수 있어요.

파이썬 코드

```
def set_age(age):
    if age < 0:
        raise ValueError("나이는 0 이상이어야 합니다")   # 예외를 직접 발생
    print(f"나이를 {age}살로 설정했어요")

try:
    set_age(20)
    set_age(-5)
except ValueError as e:
    print("오류:", e)
```

실행 결과

```
나이를 20살로 설정했어요
오류: 나이는 0 이상이어야 합니다
```

나만의 예외 만들기

파이썬 코드

```
class NotEnoughMoney(Exception):          # Exception 을 상속해서 만든다
    pass

def pay(balance, price):
    if price > balance:
        raise NotEnoughMoney(f"{price - balance:,}원이 부족해요")
    return balance - price

try:
    print(pay(10000, 3000))
    print(pay(10000, 15000))
except NotEnoughMoney as e:
    print("결제 실패:", e)
```

실행 결과

```
7000
결제 실패: 5,000원이 부족해요
```

10.5 오류에 대비한 메모장 프로그램 만들기

9장의 메모장을 업그레이드해요. 메모 파일이 없을 때, 빈 메모를 쓸 때, 메모를 지울 때 생길 수 있는 오류를 모두 처리합니다.

파이썬 코드

```
# memo_safe.py — 오류에 대비한 메모장
import os
FILE = "memo_safe.txt"
if os.path.exists(FILE):
    os.remove(FILE)                           # 실습을 위해 파일 없이 시작

def load():
    try:
        with open(FILE, encoding="utf-8") as f:
            return [line.strip() for line in f]
    except FileNotFoundError:                 # 처음 실행하면 파일이 없다
        return []

def save(memos):
    with open(FILE, "w", encoding="utf-8") as f:
        f.write("\n".join(memos) + ("\n" if memos else ""))

while True:
    menu = input("\n[1]쓰기 [2]목록 [3]삭제 [4]끝 > ")
    memos = load()
    if menu == "1":
        text = input("메모: ").strip()
        if not text:
            print("빈 메모는 저장하지 않아요.")
            continue
        memos.append(text); save(memos); print("저장!")
    elif menu == "2":
        if not memos:
            print("메모가 없어요.")
        for i, m in enumerate(memos, 1):
            print(f"{i}. {m}")
    elif menu == "3":
        try:
```

```
            n = int(input("지울 번호: "))
            removed = memos.pop(n - 1)        # 없는 번호면 IndexError
            save(memos); print(f"'{removed}' 삭제")
        except ValueError:
            print("번호는 숫자로 입력해 주세요.")
        except IndexError:
            print("그런 번호의 메모는 없어요.")
    elif menu == "4":
        print("종료합니다."); break
```

실행 결과 · 입력값: 2, 1, (Enter), 1, 장보기, 1, 운동하기, 3, 5, 3, 1, 2, 4

```
[1]쓰기 [2]목록 [3]삭제 [4]끝 > 2
메모가 없어요.

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 1
메모:
빈 메모는 저장하지 않아요.

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 1
메모: 장보기
저장!

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 1
메모: 운동하기
저장!

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 3
지울 번호: 5
그런 번호의 메모는 없어요.

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 3
지울 번호: 1
'장보기' 삭제

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 2
1. 운동하기

[1]쓰기 [2]목록 [3]삭제 [4]끝 > 4
종료합니다.
```

TIP 실행 결과를 보면 파일이 없을 때(목록), 빈 메모, 없는 번호(5) 를 모두 프로그램이 멈추지 않고 처리했어요. **사용자는 언제나 예상 밖의 입력을 한다**고 생각하고 대비하는 것이 좋은 프로그램입니다.

총정리 퀴즈

Q1. Traceback 오류 메시지를 읽을 때 가장 먼저 볼 곳은?

① 맨 윗줄
② 맨 아랫줄 (오류 종류와 이유)
③ 가운데
④ 파일 이름

Q2. 딕셔너리에 없는 키를 d["x"] 로 꺼내면 나는 오류는?

① IndexError
② KeyError
③ NameError
④ ValueError

Q3. try 에서 오류가 나지 않았을 때만 실행되는 블록은?

① except
② else
③ finally
④ raise

Q4. 오류가 나든 안 나든 항상 실행되는 블록은?

① else
② except
③ finally
④ try

Q5. 규칙에 맞지 않는 값일 때 예외를 직접 발생시키는 키워드는?

① throw
② raise
③ error
④ except

CHAPTER 11

코드를 나누고 가져오는 모듈과 라이브러리

학습 목표

- 모듈이 무엇이고 왜 필요한지 설명할 수 있다.
- import 의 여러 형태로 표준 라이브러리를 쓸 수 있다.
- 내 모듈을 만들고 다른 파일에서 가져와 쓸 수 있다.
- 패키지와 pip 로 외부 라이브러리를 설치하는 법을 안다.
- 프로그램을 여러 모듈로 나눠 정리할 수 있다.

11.1 모듈은 왜 필요할까?

프로그램이 커지면 파일 하나에 모든 코드를 넣기 어려워요. 기능별로 **파일을 나눠 두고 필요할 때 가져다 쓰면** 정리가 잘 되고, 다른 프로그램에서도 재사용할 수 있어요. 이렇게 가져다 쓸 수 있는 파이썬 파일(.py)을 **모듈**이라고 합니다.

용어	뜻	예
모듈	가져다 쓰는 .py 파일 하나	math, random, 내가 만든 calc.py
패키지	모듈을 모은 폴더	json, urllib
라이브러리	모듈·패키지 모음을 넓게 부르는 말	표준 라이브러리, pandas
표준 라이브러리	파이썬을 설치하면 함께 오는 것	math, random, datetime
외부 라이브러리	pip 로 따로 설치하는 것	requests, pandas

파이썬 코드

```
import math                     # math 모듈 가져오기
print(math.sqrt(16))            # 모듈이름.기능 으로 사용
print(math.pi)
```

실행 결과

```
4.0
3.141592653589793
```

11.2 표준 라이브러리 활용하기

import 하는 여러 방법

파이썬 코드

```
import random                   # ① 모듈 전체
from math import ceil, floor    # ② 필요한 기능만 → 이름만으로 사용
import datetime as dt           # ③ 별명 붙이기

print(random.randint(1, 6))     # 1~6 주사위 (실행할 때마다 다름)
print(ceil(3.2), floor(3.8))    # 올림, 내림
print(dt.date(2026, 12, 25) - dt.date(2026, 11, 4))   # 날짜 빼기
```

실행 결과

```
6
4 3
51 days, 0:00:00
```

자주 쓰는 표준 라이브러리 맛보기

파이썬 코드

```
import random
menu = ["김밥", "라면", "돈가스", "비빔밥"]
print("오늘 점심:", random.choice(menu))     # 하나 고르기
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)                        # 섞기
print(cards, random.sample(range(1, 46), 6)) # 겹치지 않게 6개 뽑기
```

실행 결과

```
오늘 점심: 김밥
[2, 4, 5, 3, 1] [9, 7, 35, 6, 38, 28]
```

파이썬 코드

```
import os, time
print("지금 폴더에 있는 .txt 파일 수:", len([f for f in os.listdir(".") if f.endswith(".txt")]))
start = time.time()
total = sum(range(1_000_000))
print("백만까지 더하는 데 1초도 안 걸렸나요?", time.time() - start < 1)
```

실행 결과

```
지금 폴더에 있는 .txt 파일 수: 5
백만까지 더하는 데 1초도 안 걸렸나요? True
```

TIP **부록 C** 에 자주 쓰는 표준 라이브러리 12가지를 정리해 두었어요.

11.3 모듈 만들고 사용해보기

함수를 담은 .py 파일을 만들면 그게 바로 모듈이에요. 같은 폴더에 아래 `calc.py` 를 저장해 보세요.

파일: calc.py

```
# calc.py — 계산 기능을 모아 둔 나만의 모듈
PI = 3.14159

def add(a, b):
    return a + b

def circle_area(r):
    return PI * r ** 2

if __name__ == "__main__":        # 이 파일을 직접 실행할 때만 아래가 실행된다
    print("calc.py 시험 실행:", add(1, 2))
```

같은 폴더의 다른 파일(예: main.py)에서 가져다 씁니다.

파이썬 코드

```
import calc                       # calc.py 를 가져온다 (.py 는 쓰지 않음)
print(calc.add(10, 5))
print(round(calc.circle_area(3), 2))
from calc import PI
print(PI)
```

실행 결과

```
15
28.27
3.14159
```

if __name__ == "__main__": 의 뜻

실행 방법	__name__ 의 값	아래 코드
python calc.py (직접 실행)	"__main__"	실행됨
import calc (가져다 씀)	"calc"	실행 안 됨

모듈 안에 시험용 코드를 넣어 두어도, 다른 파일에서 import 할 때는 실행되지 않게 막아 주는 장치예요.

11.4 패키지와 외부 라이브러리 알아보기

패키지 = 모듈을 모은 폴더

```
화면
shop/                ← 패키지 (폴더)
    __init__.py      ← 패키지라는 표시 (비어 있어도 됨)
    order.py         ← 모듈
    payment.py       ← 모듈
main.py

# main.py 에서
from shop import order
from shop.payment import pay
```

pip 로 외부 라이브러리 설치하기

전 세계 개발자가 만든 라이브러리는 **PyPI**(pypi.org)에 모여 있고, 명령 프롬프트에서 `pip` 로 설치해요.

```
화면
C:\> pip install requests       ← 설치
C:\> pip list                   ← 설치된 목록
C:\> pip uninstall requests     ← 삭제
```

라이브러리	하는 일
requests	웹 페이지·API 가져오기
pandas	표 데이터 분석
matplotlib	그래프 그리기
numpy	빠른 숫자 계산
flask / fastapi	웹 서버 만들기

```
파이썬 코드 (실행하지 않는 예시)
import requests                                  # pip install requests 필요
res = requests.get("https://api.github.com")
print(res.status_code)                           # 200 이면 성공
```

> **TIP** 프로젝트마다 라이브러리 버전이 다를 수 있어서, 실무에서는 **가상환경**(`python -m venv venv`)을 만들어 그 안에 설치해요.

11.5 메모장 프로그램을 모듈로 나누기

10장의 메모장을 **파일 저장 담당 모듈**과 **화면 담당 프로그램**으로 나눠 봐요. 역할이 나뉘면 고치기도, 다른 곳에서 다시 쓰기도 쉬워져요.

파일: memo_store.py

```
# memo_store.py — 메모를 파일에 읽고 쓰는 일만 담당하는 모듈
FILE = "memo_mod.txt"

def load():
    """저장된 메모 목록을 돌려준다. 파일이 없으면 빈 리스트."""
    try:
        with open(FILE, encoding="utf-8") as f:
            return [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        return []

def add(text):
    with open(FILE, "a", encoding="utf-8") as f:
        f.write(text + "\n")

def clear():
    open(FILE, "w", encoding="utf-8").close()
```

파이썬 코드

```
# memo_app.py — 화면(입력·출력)만 담당, 저장은 memo_store 에게 맡긴다
import memo_store as store
store.clear()                                  # 실습을 위해 비우고 시작

while True:
    menu = input("[1]쓰기 [2]목록 [3]끝 > ")
    if menu == "1":
        store.add(input("메모: "))
    elif menu == "2":
        for i, m in enumerate(store.load(), 1):
            print(f"  {i}. {m}")
    elif menu == "3":
        break
```

실행 결과 · 입력값: 1, 모듈 공부, 1, 패키지 공부, 2, 3

```
[1]쓰기 [2]목록 [3]끝 > 1
메모: 모듈 공부
[1]쓰기 [2]목록 [3]끝 > 1
메모: 패키지 공부
[1]쓰기 [2]목록 [3]끝 > 2
  1. 모듈 공부
  2. 패키지 공부
[1]쓰기 [2]목록 [3]끝 > 3
```

11.6 모듈과 라이브러리 연습하기

표준 라이브러리 여러 개를 함께 써서 **로또 번호 생성기**와 **D-day 계산기**를 만들어 봐요.

파이썬 코드

```
import random
from datetime import date

def lotto():
    return sorted(random.sample(range(1, 46), 6))   # 1~45 중 겹치지 않게 6개

def d_day(target):
    left = (target - date(2026, 11, 4)).days        # 오늘을 2026-11-04 로 가정
    return f"D-{left}" if left > 0 else ("D-Day" if left == 0 else f"D+{-left}")

for i in range(1, 4):
    print(f"{i}게임: {lotto()}")
print("크리스마스까지", d_day(date(2026, 12, 25)))
```

실행 결과

```
1게임: [2, 8, 15, 16, 18, 41]
2게임: [6, 7, 9, 28, 35, 38]
3게임: [2, 3, 6, 14, 15, 33]
크리스마스까지 D-51
```

파이썬 코드

```
import json                                          # 파이썬 자료 ↔ 문자열(JSON)
profile = {"name": "김파이", "skills": ["python", "git"], "age": 20}
text = json.dumps(profile, ensure_ascii=False)       # 딕셔너리 → JSON 문자열
print(text)
back = json.loads(text)                              # JSON 문자열 → 딕셔너리
print(back["skills"][0])
```

실행 결과

```
{"name": "김파이", "skills": ["python", "git"], "age": 20}
python
```

총정리 퀴즈

Q1. 다른 파일에서 가져다 쓸 수 있는 .py 파일을 무엇이라 하나요?

① 함수
② 모듈
③ 클래스
④ 변수

Q2. from math import sqrt 로 가져온 뒤 쓰는 방법은?

① math.sqrt(9)
② sqrt(9)
③ import.sqrt(9)
④ math->sqrt(9)

Q3. import numpy as np 에서 np 는?

① 새 라이브러리
② numpy 에 붙인 별명
③ 함수
④ 파일 이름

Q4. 외부 라이브러리를 설치하는 명령은?

① python install requests
② pip install requests
③ import requests
④ get requests

Q5. calc.py 를 import 할 때 실행되지 않는 부분은?

① def 로 정의한 함수
② if __name__ == "__main__": 아래 코드
③ PI = 3.14 같은 변수
④ 주석

CHAPTER 12

한 걸음 더 깊게 알아보는 파이썬

학습 목표

- 변수는 값을 가리키는 이름표라는 것과, 복사(얕은·깊은)의 차이를 안다.
- 함수를 값처럼 변수에 담고, 넘기고, 돌려줄 수 있다.
- 이터러블·이터레이터·제너레이터의 관계를 이해한다.

12.1 값을 복사한다는 것은 무엇일까?

파이썬의 변수는 값을 담은 상자라기보다, **값(객체)에 붙인 이름표**에 가까워요. `b = a` 는 값을 복사하는 게 아니라 **같은 값에 이름표를 하나 더 붙이는 것**입니다.

파이썬 코드

```
a = [1, 2, 3]
b = a                   # 같은 리스트에 이름표 b 를 하나 더
b.append(4)
print("a:", a)          # a 도 바뀌었다!
print(a is b)           # is: 같은 객체인가?
```

실행 결과

```
a: [1, 2, 3, 4]
True
```

바뀌는 값과 바뀌지 않는 값

정수·문자열·튜플은 **바꿀 수 없는(immutable)** 값이라, "바꾼다"는 건 사실 새 값에 이름표를 옮겨 붙이는 거예요. 그래서 위 같은 문제가 생기지 않아요. 리스트·딕셔너리·집합처럼 **바뀌는(mutable)** 값을 다룰 때만 주의하면 됩니다.

파이썬 코드

```
x = 10
y = x
y = y + 1               # 새 값 11 에 y 를 붙인다
print(x, y)             # x 는 그대로
```

실행 결과

```
10 11
```

진짜 복사하기: 얕은 복사와 깊은 복사

파이썬 코드

```
import copy
original = [1, [10, 20]]
shallow = original.copy()          # 얕은 복사 (= list(original), original[:])
deep = copy.deepcopy(original)     # 깊은 복사: 안쪽 리스트까지 새로

original[0] = 99                   # 바깥 값 변경
original[1].append(30)             # 안쪽 리스트 변경
print("원본:", original)
print("얕은:", shallow)            # 안쪽 리스트는 함께 바뀜!
print("깊은:", deep)               # 완전히 독립
```

실행 결과
```
원본: [99, [10, 20, 30]]
얕은: [1, [10, 20, 30]]
깊은: [1, [10, 20]]
```

방법	바깥 리스트	안쪽 리스트
b = a	공유	공유
a.copy() / a[:]	새로 만듦	공유
copy.deepcopy(a)	새로 만듦	새로 만듦

TIP 리스트 안에 리스트(2차원)가 있으면 `deepcopy` 를 쓰세요. 숫자·문자열만 담긴 리스트라면 `copy()` 로 충분해요.

함수에 리스트를 넘길 때

파이썬 코드
```
def add_item(cart, item):
    cart.append(item)           # 넘겨받은 리스트를 직접 바꾼다

my_cart = ["우유"]
add_item(my_cart, "빵")
print(my_cart)                  # 함수 밖의 리스트도 바뀐다
```

실행 결과
```
['우유', '빵']
```

12.2 함수를 값처럼 다루기

파이썬에서는 **함수도 값**이에요. 변수에 담고, 리스트에 넣고, 다른 함수에 넘기고, 함수에서 돌려줄 수 있어요. (이런 함수를 **일급 객체**라고 불러요)

파이썬 코드
```
def shout(text):
    return text.upper() + "!"

speak = shout                   # 괄호 없이 쓰면 함수 자체를 가리킨다
print(speak("hello"))
print(type(shout))
```

실행 결과
```
HELLO!
<class 'function'>
```

함수를 인수로 넘기기

파이썬 코드
```
def apply(func, values):
    return [func(v) for v in values]

print(apply(abs, [-1, -2, 3]))
print(apply(str.upper, ["a", "b"]))
print(apply(lambda x: x * x, [1, 2, 3]))
```

실행 결과
```
[1, 2, 3]
['A', 'B']
[1, 4, 9]
```

파이썬 코드
```
# 계산기: 기호와 함수를 딕셔너리에 담기
ops = {"+": lambda a, b: a + b, "-": lambda a, b: a - b,
       "*": lambda a, b: a * b, "/": lambda a, b: a / b}
for sym in ops:
    print(f"8 {sym} 2 = {ops[sym](8, 2)}")
```

실행 결과
```
8 + 2 = 10
8 - 2 = 6
8 * 2 = 16
8 / 2 = 4.0
```

함수를 돌려주는 함수 (클로저)

파이썬 코드
```
def make_multiplier(n):
    def multiply(x):
        return x * n            # 바깥 함수의 n 을 기억한다
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(10), triple(10))
```

실행 결과
```
20 30
```

데코레이터 맛보기

함수를 받아 기능을 덧붙인 새 함수를 돌려주는 함수를 **데코레이터**라고 해요. `@이름` 으로 붙입니다.

파이썬 코드
```
def log(func):
    def wrapper(*args):
        print(f"[기록] {func.__name__}{args} 호출")
        return func(*args)
    return wrapper

@log                            # add = log(add) 와 같다
def add(a, b):
    return a + b

print(add(3, 4))
```

실행 결과
```
[기록] add(3, 4) 호출
7
```

12.3 반복의 정체, 이터러블과 제너레이터

for 문으로 돌 수 있는 것(리스트, 문자열, range, 딕셔너리, 파일…)을 **이터러블(iterable)**이라고 해요. for 문은 사실 뒤에서 이런 일을 합니다.

1. `iter(이터러블)` 로 **이터레이터**(값을 하나씩 꺼내 주는 도구)를 만든다
2. `next(이터레이터)` 로 다음 값을 하나씩 꺼낸다
3. 더 꺼낼 게 없으면 `StopIteration` 이 나고, for 문이 끝난다

파이썬 코드
```
nums = [10, 20, 30]
it = iter(nums)
print(next(it))
print(next(it))
print(next(it))
print(next(it, "끝"))       # 기본값을 주면 오류 대신 그 값을 돌려준다
```

실행 결과
```
10
20
30
끝
```

제너레이터: 필요할 때 하나씩 만들어 내는 함수

함수 안에서 `return` 대신 **yield** 를 쓰면 제너레이터가 돼요. 값을 한 번에 다 만들지 않고, **요청할 때마다 하나씩** 만들어 메모리를 아낍니다.

파이썬 코드
```
def countdown(n):
    while n > 0:
        yield n                 # 값을 하나 내주고 여기서 잠시 멈춘다
        n -= 1

gen = countdown(3)
print(next(gen))
for value in gen:               # 남은 값을 이어서 꺼낸다
    print(value)
```

실행 결과
```
3
2
1
```

파이썬 코드
```
# 리스트 vs 제너레이터 표현식: 메모리 크기 비교
import sys
big_list = [n * n for n in range(100_000)]
big_gen = (n * n for n in range(100_000))      # 대괄호 대신 소괄호
print("리스트:", sys.getsizeof(big_list), "바이트")
print("제너레이터:", sys.getsizeof(big_gen), "바이트")
print("합계는 같다:", sum(big_gen) == sum(big_list))
```

실행 결과
```
리스트: 800984 바이트
제너레이터: 200 바이트
합계는 같다: True
```

TIP range() 도 값을 미리 다 만들지 않는 이터러블이에요. 그래서 `range(10**9)` 도 순식간에 만들어집니다. 큰 파일을 한 줄씩 읽는 것(9장)도 같은 원리예요.

총정리 퀴즈

Q1. a = [1, 2]; b = a; b.append(3) 후 a 는?

① [1, 2]
② [1, 2, 3]
③ [3]
④ 오류

Q2. 안쪽 리스트까지 완전히 새로 복사하는 방법은?

① a.copy()
② a[:]
③ copy.deepcopy(a)
④ b = a

Q3. f = len 다음 f("abc") 의 결과는?

① "abc"
② 3
③ len
④ 오류

Q4. 함수 안에서 값을 하나씩 내주며 멈추는 키워드는?

① return
② yield
③ next
④ break

Q5. for 문으로 돌 수 있는 것을 무엇이라 하나요?

① 데코레이터
② 이터러블
③ 클로저
④ 모듈

APPENDIX

파이썬 실력을 완성하는 바로 쓰는 알찬 부록

학습 목표

- 인공지능(AI) 도구를 공부 도우미로 똑똑하게 쓰는 법을 안다.
- 자주 쓰는 내장 함수와 표준 라이브러리를 찾아 쓸 수 있다.
- 종합 문제 20개로 전체 내용을 점검한다.

부록 A. 인공지능과 함께 파이썬 공부하기

ChatGPT, Claude, Gemini 같은 AI 도구는 **24시간 옆에 있는 과외 선생님**처럼 쓸 수 있어요. 다만 답을 받아 적기만 하면 실력이 늘지 않아요. **스스로 생각한 다음에 묻는 것**이 핵심입니다.

이렇게 물어보면 좋아요

상황	질문 예시
개념이 어려울 때	"파이썬의 딕셔너리를 중학생도 알 수 있게 비유로 설명해 줘"
오류가 났을 때	"이 코드에서 아래 오류가 났어. 원인과 고치는 방법을 알려 줘. (코드 + 오류 메시지 전체)"
코드를 이해하고 싶을 때	"이 코드를 한 줄씩 설명해 줘"
연습 문제가 필요할 때	"for 문 연습 문제 3개를 쉬운 것부터 내 줘. 정답은 내가 풀고 나서 알려 줘"
내 코드를 점검받을 때	"내가 짠 코드야. 더 읽기 쉽게 고칠 점을 알려 줘. 고친 이유도 설명해 줘"

좋은 질문의 3요소

1. **상황**: 무엇을 하려고 했는지 ("성적 평균을 구하려고 했어")
2. **자료**: 코드와 오류 메시지 **전체**를 그대로 붙여 넣기
3. **원하는 것**: 정답 코드인지, 힌트인지, 설명인지 분명히

```
화면
나는 파이썬 초보야. 리스트의 평균을 구하려고 아래처럼 썼는데 오류가 나.
정답 코드보다는 어디가 틀렸는지 힌트를 먼저 줘.

scores = ["90", "80"]
print(sum(scores) / len(scores))

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

주의 AI 도 **틀린 답을 그럴듯하게** 말할 수 있어요. 받은 코드는 반드시 직접 실행해 확인하고, 이해하지 못한 코드는 쓰지 마세요. 개인정보·비밀번호·API 키는 절대 붙여 넣지 마세요.

TIP 추천 공부법: ① 교안을 읽고 ② 예제를 직접 쳐서 실행하고 ③ 숫자를 바꿔 결과를 예상해 보고 ④ 막히면 AI 에게 힌트를 요청하고 ⑤ 퀴즈로 확인하기.

부록 B. 자주 쓰는 내장 함수 25가지

내장 함수는 import 없이 바로 쓸 수 있는 함수예요.

함수	하는 일	예 → 결과
print()	화면에 출력	print("hi") → hi
input()	키보드 입력 (문자열)	input("이름: ")
len()	길이·개수	len("abc") → 3
type()	자료형 확인	type(3) → int
int()	정수로 변환	int("7") → 7
float()	실수로 변환	float("1.5") → 1.5
str()	문자열로 변환	str(10) → "10"
list()	리스트로 변환	list("ab") → ['a', 'b']
tuple()	튜플로 변환	tuple([1, 2]) → (1, 2)
dict()	딕셔너리 만들기	dict(a=1) → {'a': 1}
set()	집합으로 변환	set([1, 1, 2]) → {1, 2}
range()	숫자 줄	list(range(3)) → [0, 1, 2]
sum()	합계	sum([1, 2, 3]) → 6
max()	최댓값	max(3, 7) → 7
min()	최솟값	min([4, 2]) → 2
abs()	절댓값	abs(-5) → 5
round()	반올림	round(3.567, 1) → 3.6
sorted()	정렬된 새 리스트	sorted([3, 1]) → [1, 3]
reversed()	거꾸로 꺼내기	list(reversed([1, 2])) → [2, 1]
enumerate()	번호와 함께	list(enumerate("ab")) → [(0, 'a'), (1, 'b')]
zip()	나란히 묶기	list(zip([1, 2], "ab")) → [(1, 'a'), (2, 'b')]
map()	모두에 함수 적용	list(map(int, ["1", "2"])) → [1, 2]
filter()	조건에 맞는 것만	list(filter(None, [0, 1, 2])) → [1, 2]
any() / all()	하나라도 참 / 모두 참	any([0, 1]) → True
isinstance()	자료형 검사	isinstance(3, int) → True

파이썬 코드

```
words = ["banana", "Apple", "cherry"]
print(sorted(words, key=str.lower))              # 대소문자 무시하고 정렬
print(list(map(len, words)))
print(any(w.startswith("A") for w in words), all(len(w) > 3 for w in words))
print(divmod(17, 5), pow(2, 5), chr(65), ord("A"))   # 덤: 몫과 나머지, 거듭제곱, 문자↔코드
```

실행 결과

```
['Apple', 'banana', 'cherry']
[6, 5, 6]
True True
(3, 2) 32 A 65
```

부록 C. 자주 쓰는 표준 라이브러리 12가지

모듈	하는 일	대표 기능
math	수학 계산	sqrt, ceil, floor, pi
random	난수	randint, choice, shuffle, sample
datetime	날짜·시간	date, datetime.now(), timedelta
time	시간 재기·기다리기	time(), sleep()
os	파일·폴더 다루기	listdir, path.exists, makedirs
sys	파이썬 실행 환경	argv, exit, version
json	JSON 변환	dumps, loads, dump, load
csv	CSV 파일 읽기·쓰기	reader, writer, DictReader
re	정규 표현식(패턴 찾기)	findall, sub, match
collections	편리한 자료형	Counter, defaultdict, deque
statistics	통계	mean, median, stdev
pathlib	경로를 객체로 다루기	Path, read_text, glob

파이썬 코드

```
import math, statistics, re
from collections import Counter
from datetime import date, timedelta

print(math.sqrt(2) ** 2 == 2, math.isclose(math.sqrt(2) ** 2, 2))   # 실수 비교는 isclose
print(statistics.mean([70, 80, 95]), statistics.median([3, 1, 2]))
print(Counter("hello world").most_common(2))                        # 가장 많이 나온 글자
print(re.findall(r"\d+", "주문 3개, 가격 4500원"))                    # 숫자만 뽑기
print(date(2026, 11, 4) + timedelta(days=100))                      # 100일 뒤
```

실행 결과

```
False True
81.66666666666667 2
[('l', 3), ('o', 2)]
['3', '4500']
2027-02-12
```

파이썬 코드

```
import csv
with open("members.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["이름", "나이"])
    writer.writerows([["김파이", 20], ["이코딩", 23]])
with open("members.csv", encoding="utf-8") as f:
    for row in csv.DictReader(f):              # 첫 줄을 키로 쓰는 딕셔너리
        print(row["이름"], int(row["나이"]) + 1)
```

실행 결과

```
김파이 21
이코딩 24
```

부록 D. 종합 실력 점검 20문제

1장부터 12장까지의 내용을 골고루 담았어요. 앱에서 풀면 바로 채점되고 해설을 볼 수 있어요. 교안에서는 맨 뒤 **정답과 해설**을 확인하세요.

문제

Q1. print(10 // 3, 10 % 3) 의 결과는?

① 3.33 1
② 3 1
③ 3 0
④ 4 1

Q2. x = "5"; y = 3 일 때 오류 없이 8 을 출력하는 코드는?

① print(x + y)
② print(int(x) + y)
③ print(x * y)
④ print(str(x) + y)

Q3. s = "Python" 에서 s[-3:] 의 결과는?

① Pyt
② hon
③ on
④ tho

Q4. 이름과 전화번호를 짝지어 저장하기에 가장 알맞은 자료형은?

① 리스트
② 튜플
③ 딕셔너리
④ 집합

Q5. len({1, 2, 2, 3}) 의 결과는?

① 4
② 3
③ 2
④ 오류

Q6. n = 15 일 때 if n % 3 == 0 and n % 5 == 0: 의 조건은?

① True
② False
③ 15
④ 오류

Q7. 다음 중 False 로 취급되는 값은?

① "0"
② [0]
③ 0
④ " "

Q8. for i in range(2, 10, 3): 에서 i 가 가지는 값은?

① 2, 3, 4, … 9
② 2, 5, 8
③ 2, 5, 8, 11
④ 3, 6, 9

Q9. [x for x in range(10) if x % 2] 의 결과는?

① [0, 2, 4, 6, 8]
② [1, 3, 5, 7, 9]
③ [2, 4, 6, 8]
④ []

Q10. while 문 안에서 이번 차례만 건너뛰고 다음 반복으로 가는 명령은?

① break
② continue
③ pass
④ return

Q11. def f(a, b=2): return a * b 일 때 f(3) 과 f(3, 4) 는?

① 3, 12
② 6, 12
③ 6, 7
④ 오류

Q12. 함수 안에서 만든 변수를 함수 밖에서 쓰면?

① 그대로 쓸 수 있다
② NameError 가 난다
③ 0 이 된다
④ 전역 변수가 된다

Q13. sorted(["bb", "a", "ccc"], key=len) 의 결과는?

① ['a', 'bb', 'ccc']
② ['ccc', 'bb', 'a']
③ ['a', 'ccc', 'bb']
④ ['bb', 'a', 'ccc']

Q14. 클래스에서 인스턴스마다 따로 가지는 값을 만드는 곳은?

① 클래스 바로 아래
② __init__ 안의 self.이름
③ 클래스 밖
④ import 문

Q15. 자식 클래스에서 부모의 __init__ 을 부르는 방법은?

① parent.__init__()
② super().__init__()
③ self.__init__()
④ base()

Q16. 파일 끝에 내용을 이어 쓰는 모드는?

① "w"
② "r"
③ "a"
④ "x"

Q17. try 블록에서 오류가 없을 때만 실행되는 블록은?

① except
② finally
③ else
④ raise

Q18. int("열") 을 실행하면 나는 오류는?

① TypeError
② ValueError
③ NameError
④ SyntaxError

Q19. import random as rd 다음 1~6 사이 정수를 뽑는 코드는?

① random.randint(1, 6)
② rd.randint(1, 6)
③ rd.random(1, 6)
④ randint(1, 6)

Q20. a = [[1], [2]]; b = a.copy(); b[0].append(9) 후 a 는?

① [[1], [2]]
② [[1, 9], [2]]
③ [[9], [2]]
④ 오류

정답과 해설

01장 총정리 퀴즈

번호	정답	해설
Q1	②	알고리즘을 프로그래밍 언어로 옮겨 적은 것이 프로그램이에요.
Q2	③	print("글자") 처럼 괄호 안의 내용을 화면에 출력해요.
Q3	①	PATH에 추가해야 명령 프롬프트 어디서든 python 명령을 쓸 수 있어요.
Q4	③	hello.py 처럼 .py 로 저장해요.
Q5	②	줄 맨 앞의 # 뒤는 주석이라 실행되지 않아요. 따옴표 안의 # 은 그냥 글자예요.

02장 총정리 퀴즈

번호	정답	해설
Q1	②	파이썬의 = 는 '대입' 이에요. 같은지 비교할 때는 == 를 써요.
Q2	③	x += 3 은 x = x + 3 과 같아요. 5 + 3 = 8
Q3	③	숫자로 시작하는 이름은 쓸 수 없어요.
Q4	②	sep 은 값 사이에 넣을 글자예요.
Q5	②	f-문자열은 {변수} 자리에 값을 넣어 줘요.

03장 총정리 퀴즈

번호	정답	해설
Q1	②	소수점이 있으면 실수(float)예요.
Q2	②	// 는 몫(3), % 는 나머지(1)예요.
Q3	②	문자열 곱하기는 반복이에요.
Q4	②	1번부터 4번 '앞'(3번)까지: y, t, h
Q5	②	str(20) 으로 바꾸거나 f-문자열을 써야 해요.
Q6	②	split 은 구분자로 나눠 리스트를 돌려줘요.

04장 총정리 퀴즈

번호	정답	해설
Q1	②	append 는 맨 뒤에 추가해요.
Q2	②	튜플은 바꿀 수 없는(immutable) 자료형이에요.
Q3	②	없는 키면 두 번째 값(0)을 돌려줘요.
Q4	②	중복이 사라져 {1, 2, 3} 이에요.
Q5	②	빈 문자열은 거짓으로 취급해요.
Q6	②	{} 는 빈 딕셔너리예요.

05장 총정리 퀴즈

번호	정답	해설
Q1	②	파이썬은 들여쓰기(보통 4칸)로 블록을 만들어요.

번호	정답	해설
Q2	②	80 이상은 거짓, 70 이상이 참이라 B 예요.
Q3	②	= 는 대입, == 는 비교예요.
Q4	②	and 는 둘 다 참일 때만 참이에요.
Q5	②	자료형은 맞지만 값이 알맞지 않으면 ValueError 예요.

06장 총정리 퀴즈

번호	정답	해설
Q1	②	조건이 거짓이 되면 멈춰요.
Q2	②	끝 번호(5)는 포함하지 않아요.
Q3	②	break 는 반복을 탈출, continue 는 이번 차례만 건너뛰어요.
Q4	②	range(3) 은 0, 1, 2 이고 각각 2배예요.
Q5	②	3 × 4 = 12번이에요.

07장 총정리 퀴즈

번호	정답	해설
Q1	②	def 함수이름(매개변수): 로 정의해요.
Q2	②	4 × 3 = 12 를 돌려줘요.
Q3	②	return 이 없으면 None 을 돌려줘요.
Q4	②	인수를 주지 않으면 기본값 1 을 써요.
Q5	②	5 를 넣어 5 + 1 = 6 이에요.
Q6	②	멈추는 조건이 없으면 RecursionError 가 나요.

08장 총정리 퀴즈

번호	정답	해설
Q1	②	클래스는 설계도, 인스턴스는 그 설계도로 만든 실제 객체예요.
Q2	②	__init__ 에서 속성을 준비해요.
Q3	②	hero.attack() 이면 self 는 hero 예요.
Q4	②	Dog 는 Animal 을 상속받은 자식 클래스예요.
Q5	③	클래스 바로 아래 만든 변수는 모든 인스턴스가 공유해요.

09장 총정리 퀴즈

번호	정답	해설
Q1	②	.2f 는 소수점 둘째 자리까지예요.
Q2	②	이어 쓰려면 "a" 를 써야 해요.
Q3	②	close() 를 잊어도 안전하게 닫혀요.
Q4	②	input() 은 문자열을 돌려줘서 이어 붙여져요. int() 로 바꿔야 해요.
Q5	③	readlines() 는 ['줄1₩n', '줄2₩n', ...] 을 돌려줘요.

10장 총정리 퀴즈

번호	정답	해설
Q1	②	마지막 줄에 오류 종류와 이유가 있어요.
Q2	②	없는 키는 KeyError 예요.
Q3	②	else 는 성공했을 때, finally 는 항상 실행돼요.
Q4	③	정리 작업에 써요.
Q5	②	raise ValueError("메시지") 처럼 써요.

11장 총정리 퀴즈

번호	정답	해설
Q1	②	.py 파일 하나가 모듈이에요.
Q2	②	필요한 기능만 가져오면 이름만으로 써요.
Q3	②	as 로 짧은 별명을 붙여요.
Q4	②	명령 프롬프트에서 pip install 로 설치해요.
Q5	②	가져다 쓸 때 __name__ 은 "calc" 라서 그 블록은 건너뛰어요.

12장 총정리 퀴즈

번호	정답	해설
Q1	②	b = a 는 같은 리스트에 이름표를 하나 더 붙인 것이라 a 도 바뀌어요.
Q2	③	얕은 복사는 안쪽 리스트를 공유해요.
Q3	②	함수도 값이라 변수에 담아 부를 수 있어요.
Q4	②	yield 를 쓰면 제너레이터 함수가 돼요.
Q5	②	리스트·문자열·range·파일 등이 이터러블이에요.

부록 D 종합 실력 점검

번호	정답	해설
Q1	②	// 는 몫 3, % 는 나머지 1 이에요. (3장)
Q2	②	문자열 "5" 를 int() 로 바꿔야 숫자 덧셈이 돼요. (3장)
Q3	②	뒤에서 세 번째부터 끝까지예요. (3장)
Q4	③	이름(키)으로 번호(값)를 찾을 수 있어요. (4장)
Q5	②	집합은 중복을 없애 {1, 2, 3} 이에요. (4장)
Q6	①	15 는 3 으로도 5 로도 나누어떨어져요. (5장)
Q7	③	숫자 0 은 거짓, 글자 "0" 과 [0] 은 비어 있지 않아 참이에요. (4·5장)
Q8	②	2 에서 시작해 3씩 커지고 10 은 포함하지 않아요. (6장)
Q9	②	x % 2 가 1(참)인 홀수만 모여요. (6장)
Q10	②	continue 는 아래 코드를 건너뛰고 조건 검사로 돌아가요. (6장)
Q11	②	f(3) 은 기본값 b=2 로 6, f(3, 4) 는 12 예요. (7장)
Q12	②	지역 변수는 함수가 끝나면 사라져요. (7장)
Q13	①	key=len 이면 길이 순으로 정렬해요. (7장)
Q14	②	self.name 처럼 만든 값이 인스턴스 변수예요. (8장)

번호	정답	해설
Q15	②	super() 는 부모 클래스를 가리켜요. (8장)
Q16	③	a 는 append(덧붙이기)예요. (9장)
Q17	③	else 는 성공했을 때만 실행돼요. (10장)
Q18	②	숫자로 바꿀 수 없는 값이라 ValueError 예요. (10장)
Q19	②	별명 rd 로 모듈을 불러요. (11장)
Q20	②	얕은 복사는 안쪽 리스트를 공유해서 a 도 바뀌어요. (12장)